UNIVERSITY OF ENGINEERING AND TECHNOLOGY

Nguyen Duc Huy

ASYMMETRIC HIERARCHICAL SEARCH WITH DENSE REWARD OPTIMIZATION FOR NEURAL THEOREM PROVING

BACHELOR THESIS

Major : Artificial Intelligence

Hanoi, 2026

UNIVERSITY OF ENGINEERING AND TECHNOLOGY

Nguyen Duc Huy

ASYMMETRIC HIERARCHICAL SEARCH WITH DENSE REWARD OPTIMIZATION FOR NEURAL THEOREM PROVING

BACHELOR THESIS

Major : Artificial Intelligence

Supervisor: **Assoc. Prof. Nguyen Phuong Thai**

Co-supervisor: **Prof. Hoang Le Truong**

Hanoi, 2026

Authorship

"I hereby declare that the work contained in this thesis is of my own and has not been previously submitted for a degree or diploma at this or any other higher education institution. To the best of my knowledge and belief, the thesis contains no materials previously published or written by another person except where due reference or acknowledgement is made."

Signature:……

Supervisor's approval

"I hereby approve that the thesis in its current form is ready for committee examination as a requirement for the Bachelor of Artificial Intelligence degree at the University of Engineering and Technology."

Signature:……

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Asst.Prof. Nguyen Phuong Thai, for his guidance, patience, and constant support throughout this thesis. His advice helped me shape the research direction, refine the technical ideas, and keep the work grounded when the problem became difficult.

I am also deeply grateful to Dr. Truong for his valuable comments, encouragement, and academic support during my study and research. His feedback helped me see the work from a broader perspective and improve the quality of this thesis.

I would like to thank all members of the lab for the discussions, suggestions, and shared working environment. Their questions and ideas made the research process more productive and much less lonely.

My deepest thanks go to my family for their unconditional love, trust, and support. I am also thankful to my friends for staying beside me, listening to me, and giving me energy during stressful periods.

Last but not least, I would like to acknowledge my own persistence throughout this journey. This thesis was completed through many difficult periods, and I am grateful that I kept going.

Asymmetric Hierarchical Search with Dense Reward Optimization for Neural Theorem Proving

Abstract

Formal theorem proving in Lean 4 remains difficult because proof search receives sparse binary feedback and must explore a large combinatorial space of tactics, intermediate lemmas, and decomposition choices. This thesis presents **GammaZero**, a hierarchical theorem-proving framework that combines local proof generation with structured skeleton decomposition inside an asymmetric AND/OR search graph. GammaZero introduces scaffold-aware subgoal solving, where each child goal is solved inside its parent Lean proof context rather than as an independent theorem. It also uses AST-guided Sorrifier patching to replace invalid proof regions with controlled placeholders for partial scoring, and dependency-aware reward computation over Lean expression trees to distinguish useful proof structure from redundant or unresolved fragments.

On 87 evaluated miniF2F-v2s test problems under a Lean 4.29-style environment, GammaZero solves 59 theorems, achieving a placeholder-free success rate of 67.82%. On the same evaluated set, the Kimina direct-sampling baseline reaches 53.28%, while APOLLO with recursive repair reaches 53.69%. An internal root-only ablation solves 41 out of 87 problems (47.13%); enabling hierarchical skeleton search solves 18 additional problems, giving a 20.69 percentage-point absolute gain attributable to the AND/OR decomposition mechanism. These results show that GammaZero's hierarchy is not merely a logging device: it closes problems that direct root-level proof generation misses, while producing structured, scored proof-search trajectories suitable for future policy optimization.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

Large language models have shown strong ability in generating natural-language mathematical reasoning and formal proof search [12, 15, 16]. However, a natural-language proof can hide many ambiguities: a step may omit assumptions, use an unstated lemma, rely on an informal transformation, or leave a gap that is accepted by the reader but not logically justified. Formal proof assistants address this problem by requiring every proof step to be checked in a precise logical system. Lean 4 is one such proof assistant [4]. In Lean, mathematical statements and proofs are represented in a dependent type theory, and a theorem is accepted only when Lean can elaborate the proof and check that it has the required type. This makes Lean a useful setting for evaluating whether a generated mathematical proof is not only plausible, but formally correct.

Recent LLM-assisted proving systems therefore use large language models to propose Lean tactics, proof fragments, or complete proofs. However, the LLM is not the final judge of correctness. Lean remains the verifier: generated proof code must follow the required syntax, use available declarations, match the expected types, and close all goals. This turns theorem proving into a search problem: the LLM proposes candidate proof steps, Lean verifies them, and the search procedure decides which proof states to explore next. At the same time, strict verification also makes proof generation fragile. A small local error, such as a missing delimiter, an unavailable theorem name, or an expression with the wrong type, can invalidate

the whole proof attempt.

Proving methods can be roughly grouped into three directions:

The first direction is whole-proof generation, where the model produces a complete proof in one pass. This is simple and can benefit from the model's long-form reasoning ability, but it is brittle: one local syntax, a declaration that is not available in the current context, or type error can cause the whole proof attempt to fail.

The second direction is tactic-level search. In Lean, a tactic transforms the current proof state into one or more new proof states. The model proposes local tactics, Lean executes and checks them, and the search procedure continues to explore the new proof states. Systems such as ReProver [16] combine tactic generation with best-first search. This gives backtracking and partial verification, but it also creates a large search space. Many different tactic sequences may be applicable at each proof state, each tactic may require different arguments or library lemmas, and every successful tactic can generate additional proof states that must be explored later. As proofs become longer, the search tree can grow combinatorially, requiring a large number of model samples and Lean executions before a useful proof path is found.

The third direction is proof decomposition. Instead of asking the model to solve the theorem directly or only propose the next tactic, the system asks the model to introduce useful intermediate steps or subgoals. This direction is motivated by the way long mathematical proofs are usually organized: a difficult theorem is reduced to smaller goals, and the final proof is assembled from those goals. Recent work such as ProD [5] and Seed-Prover [3] has explored lemma-based or sketch-based decomposition for formal theorem proving. Proof decomposition can reduce the difficulty of long-horizon reasoning and expose useful proof structure. However, decomposition also introduces new challenges. A generated subgoal may be mathematically true but irrelevant to the final proof. Large decompositions can introduce many additional subgoals, causing the search space to grow quickly. This also introduces a difficult credit assignment problem. A subgoal may be locally solvable but still provide little progress toward the original theorem. The usefulness of a decomposition is often visible only after several additional proof steps or after the final proof structure is completed.

These challenges make decomposition both promising and difficult: the system

must not only generate useful subgoals, but also decide which decompositions are worth exploring, how to verify them, and how to connect them back to the final proof.

1.2 Proposed Approach

We propose GammaZero, a hierarchical proof-search framework for Lean 4. Instead of treating generated proofs as purely correct or incorrect, the system preserves useful local structure from partially successful proof attempts and uses it during search. The model proposes local proofs and decompositions, Lean verifies them, and the search graph records which fragments are valid, which subgoals are created, and which dependencies are needed for the final proof.

We separate proof search into two complementary modes: directly solving the current goal and introducing proof structure with child subgoals. This leads to two corresponding action types:

- **Local proof actions** are complete local proof attempts intended to discharge the current goal. A local proof action is accepted only if Lean reports no remaining goals for the current node.
- **Skeleton actions** are decompositions with proof structure, such as `have`, `suffices`, `cases`, or `induction`. A skeleton action may create child subgoals together with a partial proof structure describing how those subgoals are combined to solve the final theorem.

This separation turns proof search into an AND/OR graph. OR nodes represent proof states, while action nodes represent candidate proof steps. Each action forms an AND node with two types: a local proof action usually attempts to close the current goal directly and a skeleton action may create several child subgoals. A skeleton succeeds only when all of its child subgoals are solved.

To make failed generations useful for later learning, we introduce an AST-guided Sorrifier that extends placeholder-based recovery approaches such as APOLLO [10]. The module localizes proof body failures and replaces the smallest recoverable failing region with `sorry`, producing a compilable partial script with placeholders. This patched script is not treated as a proof. It is used only to measure syntactic

and structural validity.

The resulting system records how generated proofs evolve during search: which proof states are explored, which decompositions create useful subgoals, which fragments remain valid after recovery, and which subgoals contribute to the final proof. The model proposes; Lean verifies; the search graph records alternatives and dependencies; the Sorrifier recovers local signal from invalid code; dependency analysis measures whether skeleton subgoals matter to the final proof; and final stitching reconstructs a proof candidate that Lean checks again. Only complete Lean verification determines final correctness, but partially valid generations can still guide search and provide intermediate reward signals.

1.3 Contributions

This thesis makes the following contributions:

- A hierarchical proof-search framework for Lean 4 that separates local proof generation from proof decomposition inside an AND/OR graph.
- A best-first search algorithm over Lean proof states using a priority queue and heuristic scoring.
- A scaffold method for solving subgoals inside the original parent proof context instead of treating them as independent theorems.
- A data synthesis pipeline that uses abstract syntax trees to recover partially valid Lean code, analyzes dependencies through Lean expression trees to evaluate the proof quality and converts proof search into structured trajectories with dense reward signals for future reinforcement learning.

Chapter 2

Background

2.1 Lean 4 and Formal Proof Checking

Lean 4 is an interactive theorem prover based on dependent type theory [4]. Under the propositions-as-types interpretation, a theorem is a type and a proof is a term inhabiting that type. A proof assistant verifies a theorem by elaborating user-facing syntax into typed internal terms and checking them with a trusted kernel.

This distinction between surface syntax and kernel terms is central to the design of GammaZero. The LLM generates surface-level Lean code: tactic scripts, proof blocks, local declarations, and theorem applications. Lean then elaborates that code into a lower-level expression whose type can be checked by the kernel. Many generated programs fail before they ever reach kernel checking because parsing, elaboration, type-class search, implicit argument inference, or tactic execution fails. An ATP (Automated Theorem Proving) system must therefore extract useful signal from partially successful proof attempts. A fragment may be syntactically valid but ill-typed, locally useful but incomplete, or complete only because it contains a placeholder. The framework in this thesis keeps those cases separate instead of collapsing them into a single failure label.

Lean proof scripts commonly use tactic mode:

```
theorem and_comm (P Q : Prop) :
  And P Q -> And Q P := by
```

```
intro h
exact And.intro h.right h.left
```

Each tactic transforms a proof state containing local hypotheses and a target goal. If all goals are discharged and the resulting proof term type-checks, Lean accepts the theorem. This kernel-checked execution model is crucial for AI-assisted proving: models may hallucinate, but Lean remains the verifier of final correctness.

Lean also provides placeholder mechanisms for incomplete proofs. The command `sorry` lets a proof script elaborate by inserting a placeholder proof term for an unresolved goal. This is useful during development, but it is not an accepted final proof in this thesis. Lean also has `admit`, which is treated here as a temporary placeholder used only inside scaffolds to stand for sibling subgoals that are not currently being solved. GammaZero may use these placeholders to inspect partial proof structure, but a theorem is counted as solved only when the final Lean check contains neither `sorry` nor `admit`.

Lean also makes proof search more sensitive to context. A tactic such as `exact h` is meaningful only if `h` is in scope and has a type compatible with the current goal. A rewrite tactic depends on which simp lemmas and local equalities are available. A proof by cases or induction can change the shape of the context and create several new branches. These properties make local proof-state serialization important: the model must see enough context to generate a plausible action, and the search system must remember enough context to decide whether two states are really interchangeable.

2.2 LLM-based Theorem Proving

Recent theorem-proving systems increasingly use LLMs to generate tactics, proof terms, or whole proofs [16]. Whole-proof generation is simple but fragile: one local mistake can invalidate the full output. Search-guided proving is more robust because each candidate step can be checked by Lean, and failed branches can be abandoned.

However, tactic-level search remains expensive. The model must select both tactic names and arguments, such as theorem names for `rw`, `exact`, or `apply`.

In larger proofs, the key step is often not a local tactic but the discovery of an intermediate step. This motivates methods that combine neural generation with structured search and explicit decomposition.

There is also a mismatch between how mathematicians describe proofs and how a tactic-level search system explores them. A human proof often says, "It suffices to prove an auxiliary inequality," or "We first establish a monotonicity step". But tactic search must discover the auxiliary subgoal, prove it, and connect it back to the original theorem. This can make the branching factor explode. Proof skeletons are one way to utilize the high-level reasoning: the model proposes the intermediate step explicitly, and the search system later solves the exposed subgoal.

2.3 Search-based Proof Generation

Search-guided theorem proving treats Lean as an environment. A state is the current proof state, an action is a generated proof step, and the transition is Lean execution. Common search strategies include beam search, best-first search, and Monte Carlo tree search [9, 12, 16]. These methods differ in how they prioritize frontier nodes, but they share the same core difficulty: the branching factor is large and useful reward may appear only after many decisions.

Proof decomposition naturally introduces an AND/OR search structure. OR structure represents alternative proof steps that can be applied to the current proof state. AND structure appears when a decomposition generates multiple subgoals that must all be solved for the proof to succeed. Compared with a purely linear proof trajectory, an AND/OR representation is better suited for reasoning about intermediate steps, subgoal dependencies, and hierarchical proof structure.

2.4 Reinforcement Learning and Sparse Feedback

Reinforcement learning is a promising direction for automated theorem proving because proof generation naturally involves sequential decision making [2, 8]. A system repeatedly chooses proof steps, observes the resulting proof states after Lean execution, and eventually succeeds or fails depending on whether the theorem is fully verified.

However, applying reinforcement learning to Lean proof generation remains difficult. Lean provides extremely sparse feedback: a proof is either accepted or rejected, and useful signal may appear only after many proof steps. Long proofs further increase the difficulty because local actions may influence the final outcome only much later in the search process.

Dense reward signals can help guide search and future policy optimization, but they must be designed carefully. A reward based only on syntactic validity may encourage compilable but logically irrelevant proof fragments. A useful reward mechanism should therefore reflect not only local validity, but also whether generated proof fragments contribute to the final proof structure.

Chapter 3

Literature Review

3.1 Neural Theorem Proving and Language Models

Neural theorem proving has increasingly moved from specialized symbolic heuristics toward systems in which neural models propose proof steps and a formal environment checks them. This separation is especially important in interactive theorem provers such as Lean, Isabelle, Coq, and HOL-style systems: the model may be uncertain, but the proof assistant provides a deterministic verification boundary. The practical question is therefore not whether the neural model can be trusted as a proof checker, but how to use a fallible proposal model to explore a very large discrete proof space efficiently.

Early LLM-based formal-proving systems often treated proof search as language modeling over tactic sequences. GPT-f demonstrated that a generative model trained on formal proof corpora can propose useful theorem-proving steps and that search around these proposals can solve nontrivial formal problems [12]. The central lesson is still relevant for Lean-based systems: model samples become valuable only after they are embedded in a verification loop. A raw generated proof is fragile, but a stream of candidate steps can be filtered, ranked, and combined by an external search procedure.

This thesis follows the same verifier-centered philosophy but changes the unit of search. Instead of treating every generated step as a flat continuation, GammaZero distinguishes local proof actions from skeleton actions. A local proof action is an

attempt to finish the current local state; a skeleton is a proposed decomposition that creates child subgoals. This makes the search graph closer to the mathematical structure of a proof, where an intermediate step is useful only if it is eventually proved and consumed by the final argument. The contribution is therefore not a new foundation for formal verification, but a search-and-credit structure that makes LLM proposals easier to verify, repair, score, and reuse.

3.2 Search Procedures for Formal Proofs

Search is unavoidable because formal theorem proving has sparse terminal feedback. A proof assistant usually provides a binary signal for a complete candidate: the proof either elaborates and type-checks, or it does not. Search procedures create intermediate decision points so that failed branches can be abandoned without discarding all progress. Beam search, best-first search, Monte Carlo tree search, and tree-structured proof search all instantiate this idea, but they differ in how they allocate effort across open states.

HyperTree Proof Search studies proof search as a structured exploration problem rather than a single linear sequence [9]. This is conceptually close to the AND/OR perspective used in GammaZero. When a proof step creates several subgoals, solving only one branch is insufficient: the decomposition succeeds only if every required child is solved. This hard conjunction is easy to lose in linear trajectory formulations, where partial progress on a subgoal may look better than it should. The AND/OR view makes that dependency explicit.

GammaZero differs from a pure tree-search formulation in two ways. First, it uses Lean scaffolds to keep subgoals attached to the exact parent proof in which they were created. A child goal is not simply a standalone theorem with a similar statement; it is a placeholder position inside a parent proof body. Second, GammaZero uses dense reward information from failed or partially repaired candidates. This means search is not only a solver but also a data-generation process: even branches that do not solve the root theorem may produce useful supervision about syntax survival, dependency quality, and action type.

3.3 Retrieval, Premise Selection, and Hammers

Many successful theorem-proving systems rely on retrieval or premise selection. In large mathematical libraries, the hard part is often not inventing a new proof idea but finding the relevant theorem, lemma, definition, or rewrite rule. LeanDojo introduced an environment for interacting with Lean and studied retrieval-augmented theorem proving through ReProver [16]. Its results emphasize that LLM proof search benefits from access to library context rather than relying only on the local goal.

Hammer-style systems pursue a related goal by connecting interactive proof assistants to automated theorem provers and premise-selection machinery. Thor integrates language models with external automated theorem provers, showing that neural guidance and symbolic automation can complement each other [6]. The language model can propose or guide a search, while automated tools can discharge subgoals once relevant premises have been identified. This pattern is important because it suggests that LLMs need not solve every local goal directly; they can also create structure that makes other solvers more effective.

GammaZero is compatible with retrieval and hammer-style extensions, but it does not depend on them in the current thesis. Its contribution lies one layer lower: it defines how generated actions are verified, classified, repaired, inserted into a proof graph, and assigned credit. A future version could augment local proof prompts with retrieved premises or allow skeleton prompts to request relevant lemmas, but the current architecture first establishes the correctness and reward boundaries for hierarchical search.

3.4 Sketching, Decomposition, and Informal Guidance

Several recent systems use informal reasoning or high-level sketches to guide formal proof search. Draft, Sketch, and Prove uses informal proof sketches to guide formal theorem provers, reflecting the observation that high-level mathematical plans are often easier to generate than fully elaborated formal proofs [7]. This line of work is closely related to GammaZero's skeleton actions: both approaches separate the discovery of proof structure from the final discharge of low-level subgoals.

The difference is that GammaZero makes the sketch executable as a Lean scaffold. A skeleton is not just a natural-language plan. It is a Lean proof body with target child holes, such as local declarations introduced by `have`. This gives the system a precise interface: Lean can elaborate the partial structure, report the child proof states associated with placeholders, and later verify the final stitched proof. The cost of this precision is that skeletons must obey stricter syntactic and semantic constraints than informal sketches. The benefit is that every decomposition is grounded in the same formal context as the final proof.

This distinction motivates scaffold-aware subgoal solving. If a skeleton introduces several subgoals, the system should not solve each child as if it were an independent theorem detached from its parent proof. Local declarations, binder structure, and sibling placeholders all affect what counts as a valid replacement. GammaZero therefore focuses one target `sorry` while sibling subgoals are temporarily replaced by `admit`; the model is asked to solve exactly the focused placeholder and preserve the rest of the scaffold. This design turns high-level decomposition into a verifiable editing task.

3.5 Benchmarks and Evaluation Datasets

The miniF2F benchmark has become a standard evaluation set for formal olympiad-style mathematics [17]. It is useful because it contains problems whose statements are formalized in proof assistants, enabling objective measurement of proof success. However, the benchmark also illustrates a broader challenge: formal benchmark statements may diverge from the intended informal problem, and small differences in theorem statements can substantially change difficulty. The miniF2F-Lean revisited work highlights these issues and motivates careful dataset selection and reporting [11].

ProofNet broadens the evaluation perspective by targeting undergraduate-level mathematics and autoformalization [1]. It is relevant here because a hierarchical prover must eventually operate across theorem statements that vary in style, domain, and amount of implicit context. Problems from algebra, number theory, inequalities, and elementary analysis often require different kinds of generated subgoals. A skeleton mechanism should therefore be evaluated not only by whether it solves more roots, but also by whether it proposes subgoals that are meaningful,

solvable, and used.

This thesis evaluates GammaZero primarily as a search and reward-data generation framework. The reported metrics are therefore broader than root solve rate alone. Root solve rate remains the strongest correctness metric, but the system also records generated actions, verification calls, repair rates, skeleton commitment statistics, dependency classes, and queue pruning counts. These measurements are needed because the framework is designed to generate training data for future optimization, not merely to report a single pass rate.

3.6 Autoformalization

Autoformalization studies the translation of informal mathematical statements, proofs, or problem descriptions into a formal language that can be checked by a proof assistant. This problem is adjacent to formal theorem proving but not identical to it. A prover starts from a formal statement and attempts to construct a proof term or tactic script. An autoformalizer may instead start from natural language and must decide which definitions, coercions, quantifier order, side conditions, and library notions capture the intended mathematics. A formally verified proof of the wrong statement is still a verification artifact, but it does not solve the original informal problem.

Large language models have made autoformalization more plausible because they can map between informal mathematical prose and formal syntax when given sufficient examples. Wu et al. study autoformalization with large language models and show that translation quality can improve when informal and formal corpora are used together [14]. The central difficulty is semantic alignment: the model must preserve the meaning of the informal statement while selecting a formal encoding accepted by the target proof assistant. This is especially delicate in mathematics because many informal statements suppress assumptions that a formal theorem must make explicit.

ProofNet highlights this distinction by pairing undergraduate-level mathematical problems with formal counterparts and by evaluating both autoformalization and proving [1]. Such datasets are valuable because they expose two separate failure modes. A system may fail before proof search begins because the formal statement is missing a hypothesis or uses an unsuitable definition. It may also receive a cor-

rect formal statement but fail to prove it. The miniF2F-Lean revisited analysis similarly motivates caution when interpreting formal benchmark results: the exact formal statement and library context can alter the problem being measured [11].

GammaZero does not attempt autoformalization in this thesis. The input is assumed to be an existing Lean theorem statement, and the system's responsibility begins after that statement is fixed. This boundary is important for evaluation. A failed GammaZero run should be interpreted as a failure to find a verified proof within the current search limits, not as evidence that the informal theorem is false or that the formalization is faithful. Conversely, a successful run certifies the formal theorem that Lean checked; it does not by itself validate an upstream natural-language translation. Future systems could connect an autoformalizer to GammaZero, but the correctness boundary would still need to distinguish translation validity from proof validity.

3.7 Reinforcement Learning for Theorem Proving

Reinforcement learning is attractive for theorem proving because proof construction is naturally sequential. A prover chooses an action, observes a new state from the proof assistant, and eventually receives a success or failure signal. Earlier work on reinforcement learning of theorem proving explored how learned guidance can improve symbolic proof search and how proof attempts can produce training signal for subsequent attempts [8]. HOList made this direction more systematic by providing an environment for machine learning over Higher Order Logic theorem proving, with explicit states, actions, and goals suitable for learned policies [2].

The main obstacle is reward sparsity. A theorem prover may need many local decisions before the final checker accepts the proof. Most intermediate states do not come with a natural scalar value, and failed branches may still contain useful mathematical structure. This makes a naive terminal reward difficult to use: the system learns only that a complete trajectory failed, without knowing whether the early plan was bad, the final tactic was malformed, or a promising subgoal was simply not finished in time. Proof-search systems therefore often combine policy guidance with search, replay, or additional value estimates rather than relying on single sampled trajectories.

Recent neural provers continue this pattern. GPT-f uses a generative model and

proof search rather than trusting one direct completion [12]. DeepSeek-Prover-V1.5 explicitly combines proof assistant feedback with reinforcement learning and Monte Carlo tree search, showing the importance of using checker feedback as a learning signal rather than treating the language model as a standalone reasoner [15]. These systems support a broader lesson: reinforcement learning for formal proving is most credible when it is grounded in verified transitions from a proof assistant.

This thesis does not train a reinforcement learning policy. Instead, GammaZero is designed to generate RL-compatible scored trajectories. Each verified, repaired, failed, or partially useful action can be recorded with environment feedback, dependency information, graph position, and final proof outcome. The goal is to make future optimization more data-rich: a policy should be able to learn not only which actions solved a root theorem, but also which skeletons introduced useful child proof states, which local tactics survived Lean checking, and which declarations were unused in the final proof.

3.8 Positioning of GammaZero

Existing work suggests several complementary directions for LLM-assisted theorem proving. Search-based systems treat Lean as a verification environment and explore many candidate proof steps. Sketch-based systems introduce higher-level proof structure through intermediate steps or informal plans. Retrieval-based systems improve local reasoning by exposing relevant library context. Autoformalization systems address the upstream problem of producing faithful formal statements from informal mathematics. Reinforcement learning systems study how proof-assistant feedback can improve future proposal policies.

GammaZero is positioned between flat tactic search and high-level proof sketching. Instead of generating one tactic at a time, the framework separates local proof generation from explicit proof decomposition. Local proof actions attempt to close the current proof state directly, while skeleton actions introduce proof structure through executable Lean scaffolds with child subgoals.

This separation changes the role of search. The search graph does not expand after every tactic step. New proof states are introduced mainly through decomposition decisions, which makes the growth of the search space depend more on skeleton structure than on low-level tactic exploration. At the same time, every

decomposition remains grounded in Lean through elaborated scaffolds and verified child proof states.

GammaZero also differs from purely success-driven proof search. Failed or partially valid generations are not discarded immediately. Instead, the framework extracts repair information, dependency structure, and local verification signals to produce structured search trajectories for future reinforcement learning and policy optimization. In that sense, GammaZero occupies a deliberately narrow but useful position: it assumes the theorem is already formalized, keeps Lean as the correctness authority, uses best-first search to spend effort on promising open states, and records dense feedback that can support future policy learning.

Chapter 4

Problem Formulation

4.1 Proof States and Proof Scaffolds

Given a theorem statement T, the objective is to construct Lean code that proves T without using `sorry`, `admit`, or any temporary placeholder. Lean verification is the final correctness criterion: a proof is accepted only when the complete theorem is checked successfully.

GammaZero still uses proof states as the basic units of search. A Lean proof state provides the local context and target goal that the model must reason about. These remain part of the state representation. However, local context and goal alone are not enough for GammaZero, because they do not fully record where the goal sits inside the parent proof or how a completed child proof will be inserted back. This distinction is important because the same goal may occur in different parts of a proof and may have access to different local facts.

To preserve this missing structure, GammaZero adds a proof scaffold around each proof state. A scaffold is a partial Lean proof that contains the current hole to be solved, the surrounding proof code, and the structure that connects this hole back to the parent goal. The model is therefore not asked to solve a child goal as an independent theorem. Instead, it solves the goal inside the proof structure where the goal was created.

Each open state is assigned one target hole. The task of the state is to replace that hole with valid Lean code. Other unfinished holes in the same scaffold are

treated as sibling holes. They are not solved by the current state; they belong to other states in the graph. This gives each state a clear local task while still keeping the full parent proof structure visible.

This design is especially important for skeleton actions. A skeleton action may introduce several child subgoals, but it also provides a partial proof structure showing how those subgoals are intended to support the parent goal. Solving the children separately is not enough by itself. Their proofs must later fit back into the scaffold so that the whole parent proof can be checked by Lean.

At a high level, a scaffolded proof state can be viewed as

$$s = (C, i, \Gamma, G),$$

where C is the proof scaffold, i identifies the target hole inside that scaffold, Γ is the local context, and G is the target goal. Lean provides Γ and G at the target hole, and these are shown to the model to make the local task explicit. However, the state is not represented only by (Γ, G). The surrounding scaffold and target position are also part of the state because they record where the hole occurs and how the completed proof will connect back to the parent proof. This distinction matters because two holes may have the same visible goal but occur in different scaffolds. A proof that is valid for one hole is not necessarily valid for the other. This gives each state a simple rule: it may replace the target hole with Lean code, but it must leave the surrounding scaffold unchanged.

After the required child states are solved, GammaZero inserts their proof bodies back into the parent scaffold and asks Lean to verify the resulting proof again. Temporary placeholders used during search do not count as proof. They are only a way to check one part of a larger proof while the other parts are still being solved. Final success always requires complete Lean verification without placeholders.

4.2 Action Space: Local Proof Actions and Skeleton Actions

GammaZero separates two ways of continuing proof search. The first way is to solve the current proof state directly. The second way is to introduce proof structure

that may create smaller subgoals. This gives two action types:

$$\mathcal{A} = \mathcal{A}_{local} \cup \mathcal{A}_{skel}.$$

Local proof actions. A local proof action is a generated Lean proof body for the current target hole. Its goal is to close the current proof state inside the current scaffold. The action may contain multiple tactics and may temporarily create subgoals during execution. However, it is accepted as a successful local proof action only if all of these goals are solved by the end of the generated proof body.

If a local proof action leaves any unresolved goal, it fails as a local proof action. It is not converted into a decomposition and it does not create child proof states in the search graph. This rule keeps the meaning of local proof actions simple: they either solve the current state or fail at that state.

This design reduces the number of search decisions inside one proof state. In a tactic-level search, each small tactic step may become a new search decision. In GammaZero, a local proof action represents a complete local attempt. Since it does not expand the graph, the growth of the graph is controlled mainly by skeleton actions.

Skeleton actions. A skeleton action is a generated Lean proof body that introduces proof structure with possible child subgoals. It may use constructs such as `have`, `suffices`, `cases`, or `induction`. Unlike a local proof action, a skeleton action is allowed to create child subgoals.

A skeleton action does not merely list subgoals. It also describes how the resulting proof pieces are intended to support the current goal. For example:

Skeleton Example

```
intro hp
have hq : Q := by
  sorry
exact h2 hq
```

The statement Q becomes a child subgoal, while the last line shows how the completed proof of Q will be used to continue the parent proof.

A skeleton action may create zero or more child subgoals. If it creates no child subgoal and the current state is solved, it is simply a successful action. If it creates child subgoals, then all of them must eventually be solved for the skeleton to succeed. In the search graph, the child subgoals become child proof states.

This gives the graph an AND/OR structure. OR nodes are proof states, because a proof state can be solved by any successful outgoing action. Skeleton actions have AND semantics, because every child proof state created by the skeleton must be solved. This structure matches proof decomposition: a decomposition is useful only when all required subgoals can be completed and connected back to the parent proof.

The concrete running example for these definitions is moved to Appendix A.

4.3 AND/OR Graph Semantics

The action definitions above induce an AND/OR graph over scaffolded proof states. OR nodes are proof states: a state is solved when at least one outgoing action succeeds. Action nodes represent generated proof attempts. A local proof action is an action with no child proof state after success. A skeleton action may have child proof states, and it succeeds only when all required children are solved and the completed scaffold is accepted by Lean.

The graph makes two different relations explicit. A proof state represents a choice among candidate actions: any successful action can solve the state. A skeleton represents a decomposition: every child proof state created by the skeleton must be solved before the skeleton can solve its parent state.

GammaZero uses a graph rather than a tree because duplicate states can be shared. However, state identity is scaffold-sensitive. Two states are merged only when they refer to the same target problem inside the same proof scaffold. Thus, a child subgoal is not merged back into an ancestor merely because the visible goal text is the same. The scaffold records where the goal occurs and how it connects to the parent proof, so equal-looking goals in different scaffolds remain distinct.

Solved status propagates through this graph. When a child proof state is solved, its parent skeleton may become solved. When a skeleton action is solved, its parent proof state may become solved. This propagation can continue upward until the

root theorem is solved.

This graph view is also used later for reward backup. OR nodes select among alternative actions, while skeleton actions are limited by their unfinished children. The full reward formulas are defined in Section 6; here the important point is only the logical dependency: alternatives are OR choices, and decomposition children are AND requirements.

4.4 Search Limits and Failure Labels

GammaZero runs proof search under finite limits. These limits include the number of generated actions, the maximum search depth, the number of local proof attempts and skeleton attempts per state, the number of open states kept in the queue, model generation limits, and Lean verification timeouts. These limits are necessary because both model sampling and Lean verification are expensive.

Because the search is finite, the labels used in the graph are operational labels. They describe what happened during the current run; they do not describe mathematical truth. In particular, failing to solve a state within the current limits does not mean that the goal is false or impossible to prove.

The main status labels are:

Status	**Meaning**
Solved	The state or action has been closed by Lean without rema
Open	The state is still active and may be expanded later.
Failed	The action, state, or skeleton cannot continue successfully
Unresolved Within Limits	The search stopped before the open state was solved.

A local proof action is marked successful only if Lean verifies that the current target has been fully solved. If it leaves an unresolved goal, it fails as a local proof action. The failed action may still be kept in the record for reward computation or analysis, but it does not create child proof states and does not solve the parent state.

A skeleton action has a stricter success condition. If it creates child proof states, then every child must be solved and the completed scaffold must be accepted by

Lean. If one required child remains unsolved when the search gives up on that decomposition, the skeleton is marked failed in the graph. This does not mean that the child goal is mathematically impossible. It only means that the skeleton was not completed under the current search limits.

A proof state is solved when at least one outgoing action succeeds. A proof state may remain open while there are still allowed attempts or while one of its active decompositions is still being explored. If all useful attempts have failed or the state has exhausted its allowed search budget, it may be marked failed for the current run. If the global search stops while the state is still open, it is recorded as unresolved within limits.

This distinction is important for reward assignment. A repaired candidate may provide useful syntactic reward signal, but it cannot solve a state if placeholders remain. A skeleton with one solved child and one unresolved child may contain useful structure, but it is not a solved decomposition. The system may keep syntactic rewards and diagnostic information from such actions, while withholding delayed structural credit until the required proof states are actually solved.

Timeouts and system failures are treated in the same operational way. They are recorded as failed actions or interrupted checks for the current run, not as evidence about the truth of the theorem. The final correctness boundary remains Lean verification of the completed proof: the root theorem is solved only when Lean accepts the fully assembled proof without `sorry`, `admit`, or any temporary placeholder.

4.5 Design Invariants

The formulation is governed by a small set of invariants. First, every scaffolded proof state has exactly one target hole. This target is the only location that the current state is responsible for solving. Other unfinished holes in the same scaffold belong to sibling states.

Second, while one target is being solved, sibling subgoals may be represented by temporary `admit` placeholders. These placeholders are part of the surrounding scaffold and must not be changed by the candidate action. The current action may replace only the target hole.

Third, local proof actions are terminal with respect to the search graph. They either close the current target or remain failed attempts recorded at the same state; they never introduce child proof states.

Fourth, a skeleton action may create child proof states by extending the current scaffold with child subgoals. If it creates child states, the skeleton cannot solve its parent until every required child has been solved and the completed scaffold passes Lean verification. If a skeleton creates no child state, it is solved only when Lean verifies that it directly closes the current target.

Fifth, a proof state can be marked SOLVED only through Lean verification. For a local state, the replacement for the target hole must not contain `sorry`, `admit`, or any temporary placeholder. For a parent skeleton, all child proof bodies must be stitched back into the parent scaffold and the completed block must be checked again.

The global invariant is that temporary placeholders are never part of an accepted theorem. They are used only to isolate responsibilities during search. A repaired script that still contains placeholders may provide reward information, but it cannot solve a state or appear in the final proof. The root theorem is accepted only after recursive assembly reaches the root and Lean verifies the complete theorem without unresolved placeholders.

These invariants also clarify how failure is represented. Parser errors, elaboration errors, timeouts, and policy-check rejections are failed actions, not failed theorems. An open state that is not solved within the current search limits remains open from the mathematical point of view, even if the exported record labels it as UNRESOLVED WITHIN LIMITS. This distinction is important for future training: the model should learn that the explored branch did not succeed under the current search configuration, but it should not learn that the underlying goal is false.

Chapter 5

AST-Guided Sorrifier

5.1 Motivation

Lean failures are often local. A generated proof may contain a valid prefix and one broken command, but normal binary verification rejects the whole script. The Sorrifier recovers partial signal by replacing localized failing regions with `sorry`.

We distinguish two notions:

- **Placeholder-compilable script:** Lean accepts the script only because unresolved or invalid regions were replaced with `sorry`.
- **Fully verified proof:** Lean accepts the script with no `sorry` placeholder.

The Sorrifier is used only for reward estimation and partial-credit extraction. A theorem is considered solved only when Lean accepts the proof without any `sorry`.

5.2 Failure Taxonomy

Generated Lean failures are categorized conceptually as follows:

- **Parser errors:** malformed syntax, such as missing brackets, invalid tactic syntax, or broken proof blocks.

- **Elaboration errors:** unresolved names, missing implicit arguments, or expressions that Lean cannot elaborate.
- **Type errors:** syntactically valid terms whose inferred types do not match the expected goal.
- **Unsolved goals:** valid proof fragments that execute but leave remaining subgoals.

This taxonomy is used to explain the main failure modes observed in generated Lean code. The current implementation does not rely on a complete classifier that formally separates parser, elaboration, and type-checking failures. Instead, candidate actions are executed through a persistent Lean worker, while an AST daemon provides syntax spans when a parse tree is available. The worker returns a structured result containing diagnostics and proof-state metadata. The main returned fields are `errors`, `warnings`, `infos`, and `sorries`. Each diagnostic contains a severity level, a textual message, and a source position:

```
{
  "pos": {"line": ..., "column": ...},
  "endPos": {"line": ..., "column": ...},
  "severity": "error" | "warning" | "info",
  "data": "..."
}
```

Operationally, the implementation performs only a coarse separation between unresolved-goal cases and fatal failures. Diagnostics indicating remaining goals are handled together with the `sorries` field, which provides the proof states associated with placeholder locations. Other diagnostic failures are passed to the Sorrifier as recoverable fatal errors. Worker crashes, system failures, and timeouts are not repair targets; they are logged as failed actions.

The recovery pipeline is therefore diagnostic-driven rather than taxonomy-driven. It starts from the first Lean error location, or from the first unsolved-goal location when the candidate elaborates but does not close the expected target. Given this source position, the Sorrifier attempts to identify the affected region and replace it with a placeholder. If an AST can be constructed, the system selects the smallest enclosing tactic, tactic sequence, expression, or declaration block that covers the

reported source position. If the AST is only partial or cannot be constructed due to severe syntax failure, the system falls back to a coarser line-, indentation-, or block-based recovery strategy. This fallback searches for nearby structural boundaries such as `have`, bullet blocks, or `:= by` proof bodies.

Different patch units are used depending on how precisely the failure can be localized. Local errors may be patched at the level of a single tactic line or AST node. Repeated failures or severe malformed blocks may trigger a larger structural patch that removes or replaces an enclosing command block. If the failure occurs in a declaration body and local recovery is unreliable, the system may replace the body with a placeholder proof:

```
:= by
  sorry
```

Unsolved goals are handled differently from fatal failures. For skeleton actions, unresolved goals reported through `sorries` are converted into child OR nodes in the AND/OR graph. For local proof actions, remaining goals mean that the action did not discharge the current node. Such actions may still receive syntactic recovery reward, but they are not allowed to expand the graph. A more fine-grained classifier for parser, elaboration, and type errors is left as a possible implementation refinement.

5.3 AST-based Error Localization

When Lean reports an error source span for a generated script, the Sorrifier maps that span to the narrowest enclosing syntax node in the script's AST. The preferred repair units are a tactic, a tactic sequence, an expression, or a declaration proof body. The system patches the smallest viable unit first, preserving the largest possible amount of surrounding generated code.

If the error belongs to a declaration body, the body can be replaced by a placeholder proof:

```
:= by
  sorry
```

If the error belongs to a local tactic or expression, only that local region is replaced where possible.

Iterative Patching Algorithm The recovery loop operates as follows:

1. Submit the generated script to Lean for elaboration and verification.
2. If the proof is fully verified without placeholders, return full success.
3. If Lean reports a syntax error, elaboration error, type error, or unsolved-goal span, locate the smallest enclosing `Syntax` node or structural block for the diagnostic coordinates.
4. Replace the failing node with `sorry` or a placeholder proof block.
5. Re-submit the patched script to Lean.
6. Stop when the script becomes placeholder-compilable or the patch budget is exhausted.

To guarantee termination and avoid cyclic oscillation, the implementation stores hashes of intermediate patched scripts. If a repeated broken state is detected, the algorithm escalates the patch boundary to a larger enclosing structural block, such as an entire `have` statement or tactic block. If no reliable local repair remains, the action is marked as failed for search while retaining diagnostics for offline analysis.

The iterative nature of the algorithm matters because Lean errors often cascade. A single type mismatch or missing argument can make later lines elaborate in the wrong local context, and an unknown identifier can cause subsequent tactics to report misleading errors. Patching only the first reported error in one pass is therefore not always enough. GammaZero repeatedly submits the patched candidate to Lean so that each next diagnostic is computed in the updated proof state. This makes the recovery process conservative: each structural replacement is justified by an active Lean response rather than by a purely textual guess about which lines look suspicious.

Crucially, the repair loop is not allowed to become a proof search procedure of its own. It does not synthesize new lemmas, search for alternative theorem bindings, or replace broken lines with semantically different tactics. Its role is to preserve the largest checkable prefix and surrounding structure by isolating failures behind placeholders. This keeps the reward interpretation simple. A high syntactic score means that much of the model's original code remained Lean-checkable after local placeholder insertion; it does not mean that the repair loop invented a new proof path.

5.4 Syntactic Survival Signal

After recovery, the system estimates how much of the original generated proof body survives in the patched, placeholder-compilable script. Although AST information may be used to localize failing regions during recovery, the current reward implementation does not count AST nodes directly. Instead, it uses a line-based syntactic score over cleaned proof lines.

The proof body is normalized by removing blank lines and comment-only lines. The remaining lines from the original generation are compared with the lines in the patched script using sequence matching. Lines that remain unchanged and in the same relative order are counted as surviving. Lines removed by patching, deletion, or orphan-code cleanup are treated as non-surviving.

This produces a syntactic signal: generations that require only small localized patches receive higher syntactic reward, while generations whose proof bodies must be mostly replaced by `sorry` receive lower syntactic reward. The exact formula, including the penalty for unused or ineffective code diagnostics, is defined in Section 6.

The line-based score is deliberately coarse. Counting AST nodes would make the reward depend on parser details rather than on the proof text produced by the model. Counting surviving lines makes the reward easier to inspect: the recorded trajectory shows exactly which generated lines were preserved and which lines were replaced by placeholders. A short proof whose only line fails should receive little syntactic credit; a longer skeleton that contains several valid declarations before a localized failure should receive more. This is not a proof-quality metric, but it is useful feedback for future optimization because it distinguishes completely

unusable generations from generations that are nearly well formed.

There are also cases where recovery is intentionally disabled. Timeouts, worker crashes, and system-level failures do not identify a trustworthy local source span, so replacing text with `sorry` would create a misleading syntactic reward signal. Similarly, action-policy violations such as using `admit` in a skeleton are not treated as ordinary Lean failures. They indicate that the model did not follow the action requirement and are recorded as rejected candidates.

A limitation of the syntactic score is that it does not measure semantic importance. A single surviving line might introduce a crucial subgoal, while several surviving lines might only set up unused local names. This is why r_{env} is later paired with the dependency reward r_{dep}, rather than used alone. The syntactic reward answers a narrow question: how much generated Lean structure survived recovery? The downstream dependency reward r_{dep} answers a later question: did that structure matter after the proof was stitched and elaborated? Keeping these rewards separate makes the trajectory more interpretable.

Another limitation is that recovery may overestimate low-quality generations. If a long invalid proof contains a few surviving fragments, the line-based score can still assign nonzero reward even when the overall proof direction is poor. A model that repeatedly emits long invalid scripts may still obtain nonzero syntactic rewards if some prefixes parse. For this reason, failed actions do not solve states and repaired skeletons receive a mild priority penalty. The search can learn from repaired code, but it still prefers clean actions when other signals are similar.

Concrete Sorrifier examples and line-based reward calculations are moved to Appendix B.

Chapter 6

Dense Reward and Credit Assignment

6.1 Syntactic Reward

For an action a generated at state s, let $L_{\text{orig}}(a)$ be the list of clean proof lines in the original generated action, excluding blank lines and comment-only lines. Let $L_{\text{patch}}(a)$ be the clean proof lines after Sorrifier recovery. The system compares these two lists using sequence matching. A line is counted as surviving only if it appears unchanged and in the same relative order in the patched proof.

Let $N_{\text{orig}} = |L_{\text{orig}}(a)|$, and let N_{surv} be the number of surviving lines. After the patched script is checked by Lean, the system also counts how many surviving lines correspond to unused or ineffective code. We denote this number by N_{dead}. These are lines that survived the recovery process but do not appear to contribute useful executable proof structure, for example because Lean reports that a declaration is unused or that a tactic has no effect. By definition, $0 \leq N_{\text{dead}} \leq N_{\text{surv}}$.

The syntactic reward is:

$$r_{\text{env}}(s, a) = \begin{cases} \dfrac{N_{\text{surv}} - N_{\text{dead}}}{N_{\text{orig}}}, & N_{\text{orig}} > 0, \\ 0, & N_{\text{orig}} = 0. \end{cases}$$

This reward measures how much of the generated Lean code remains usable

after recovery. Lines removed or replaced by placeholders do not receive credit. Surviving lines that are later identified as unused or ineffective are also discounted. Therefore, r_{env} is a syntactic validity signal, not a proof correctness signal. A high value means that much of the generated code survived Lean checking after patching; it does not mean that the action solves the proof state.

6.2 Dependency-aware Structural Reward

Syntactic validity alone is not enough to judge the quality of a generated action. Both skeleton actions and local proof actions may introduce local declarations through constructs such as `have` or `let`. In a skeleton action, these declarations often correspond to child subgoals that are solved by child search branches and stitched back into the parent scaffold. In a local proof action, they are proof steps inside a single generated proof body.

In either case, the model may introduce declarations that compile but do not help the final proof. It may also leave declarations containing placeholders, which makes the resulting proof incomplete. To estimate whether generated declarations actually contribute to the proof that was found, GammaZero performs dependency analysis over the elaborated Lean proof expression.

This signal is more semantic than the syntactic reward in Section 6.1: it is computed from Lean's elaborated expression tree rather than from surviving source lines. It therefore asks whether a declaration is used by the checked proof term, not merely whether the text around that declaration compiled. It is still not a theorem about mathematical necessity or proof minimality; it is an attribution signal for the concrete proof found by the search.

A separate expression-dumping process exports the Lean 4 expression tree into JSON. The dependency analyzer traverses nodes such as `bvar`, `fvar`, `app`, `lam`, `forallE`, and `letE` to determine whether each generated declaration is referenced by the final proof expression. The traversal is De Bruijn-aware: when the analyzer enters a binder such as `lam`, `forallE`, or `letE`, it shifts the target De Bruijn index so that bound-variable references are attributed to the correct declaration. The analyzer also detects `sorryAx` in the expression tree.

Combining usage detection with placeholder detection gives four structural cases:

- **Used and solved:** the declaration has a solved proof body and is used by the elaborated proof. This is the core useful case.
- **Unresolved and used:** the declaration still contains a placeholder and is used by the proof, so the proof is incomplete.
- **Solved but unused:** the declaration is complete but redundant for the selected proof.
- **Unresolved and unused:** the declaration is unfinished and unused by the selected proof.

Let $n_{\text{used,solved}}$, $n_{\text{unused,solved}}$, and $n_{\text{unused,unresolved}}$ denote the numbers of used-and-solved, solved-but-unused, and unresolved-and-unused declarations. The general dependency reward is:

$$r_{\text{dep}}(s,a) = \frac{n_{\text{used,solved}}}{n_{\text{used,solved}} + w_{\text{unused,solved}} n_{\text{unused,solved}} + w_{\text{unused,unresolved}} n_{\text{unused,unresolved}}},$$

where $w_{\text{unused,solved}} \geq 0$ penalizes solved but unused work and $w_{\text{unused,unresolved}} > w_{\text{unused,solved}}$ penalizes unresolved unused work more strongly. If the denominator is zero and the action introduces no local declarations, the reward defaults to 1.0. Unresolved-and-used declarations are treated as structural failures because the final proof still depends on a placeholder-containing declaration.

The formula is only a heuristic for ranking search actions. It favors used-and-solved declarations and penalizes unused or unfinished ones. The numerator counts declarations that are both used and solved. This conjunction is essential: a declaration is not useful for the final proof merely because it is solved, and it is not acceptable merely because it is used. It must be both. The denominator counts those declarations plus weighted forms of waste. A solved-but-unused declaration is mildly harmful because it consumes search effort but does not block final verification. An unresolved-and-unused declaration is worse because it leaves unfinished code in a placeholder-compilable script. The dependency score therefore helps distinguish compact useful structure from redundant or bloated proof scripts, while final correctness still depends only on Lean verification without placeholders.

This analysis is tied to the proof that GammaZero actually found. If a declaration is unused in the selected stitched proof, that does not mean it is mathematically useless in every possible proof. It only means that this search trajectory did

not need it.

6.3 AND/OR Backup

After search stops or the root is closed, GammaZero backs up values through the explored AND/OR graph. States are OR nodes: a state can be solved by choosing one successful outgoing action. Actions are AND nodes: a skeleton action succeeds only when all of its child proof states are solved.

The implemented backup uses a single action-value rule for both local proof actions and skeleton actions. Solved actions are credited through the dependency reward and, for skeletons, through solved child values.

For an action a applied to state s, let $\mathcal{C}(a)$ be the set of child states created by the action. For a local proof action, $\mathcal{C}(a) = \emptyset$. We define the empty-child future value to be zero:

$$\min_{s' \in \emptyset} V(s') = 0.$$

The action value is:

$$Q(s,a) = r_{\text{env}}(s,a) + r_{\text{dep}}(s,a) + \mathbb{1}_{\text{solved}}(s,a)\,\gamma \min_{s' \in \mathcal{C}(a)} V(s').$$

Here, r_{env} is the syntactic preservation reward from the Sorrifier, and r_{dep} is the dependency reward from the elaborated proof expression. The indicator $\mathbb{1}_{\text{solved}}(s,a)$ is 1 only when the action solves its required proof state. For a local proof action, this means the current target is closed without `sorry` or `admit`. For a skeleton action, this means every child state in $\mathcal{C}(a)$ is solved.

This rule has two useful special cases. For a local proof action, the child set is empty, so the future term drops out:

$$Q(s,a) = r_{\text{env}}(s,a) + r_{\text{dep}}(s,a).$$

For a skeleton action that is not fully solved, the solved indicator is zero, so the action keeps only its local credit. In practice, r_{dep} is zero when the skeleton has not produced a solved dependency structure. When the skeleton is solved, it also

receives the discounted value of its weakest child branch:

$$Q(s,a) = r_{\text{env}}(s,a) + r_{\text{dep}}(s,a) + \gamma \min_{s' \in \mathcal{C}(a)} V(s').$$

The min operator implements the AND part of the graph: a decomposition is limited by its weakest child state. This prevents the backup from giving high delayed value to a skeleton that solves only easy subgoals while leaving an essential one open.

For an OR node s, the state value is the best available outgoing action:

$$V(s) = \max_{a \in \mathcal{A}(s)} Q(s,a),$$

where $\mathcal{A}(s)$ is the set of actions expanded from s. The graph therefore uses max backup over alternative actions and min backup over the conjunctive children of a skeleton action.

6.4 Reward Computation and Training Data Generation

The implemented system produces scored proof-search trajectories rather than an end-to-end optimized policy checkpoint. Each sampled action is executed, optionally recovered, inserted into the AND/OR graph, and assigned syntactic and delayed structural rewards according to the formulations above. The resulting records contain the serialized proof state, action type, generated Lean code, recovery result, child proof states, graph status, priority-queue metadata, skeleton commitment metadata, and final action value.

These records can be used as training data for later policy optimization. Local proof actions and skeleton actions should be grouped separately because their value is observed through different graph structures: local proof actions have no child value term, whereas skeleton actions receive delayed child value only when all generated child subgoals are solved. Therefore, comparing local proof actions and skeleton actions in the same optimization group would mix different reward scales.

6.5 Trajectory Export and Final Verification

For a successful root theorem, GammaZero reconstructs the proof by recursively inserting solved child proof bodies into the child subgoals of the committed parent skeletons. Reserved skeletons and failed actions remain in the search record, but they are not part of the selected final proof. After insertion, dependency analysis may remove solved declarations that are unused by the elaborated proof expression. The resulting theorem is submitted to Lean again. The theorem is accepted as solved only if this final verification succeeds without real `sorry` or `admit`.

The same trajectory records can later support policy optimization, but this thesis stops at verified search, reward computation, and data export.

Chapter 7

Proposed Framework

7.1 System Components

GammaZero is organized around four interacting components:

- **Model interface:** asks the LLM for local proof actions or skeleton actions under separate output requirements.
- **Lean verification interface:** checks generated candidates, reports diagnostics, returns placeholder states, and rejects candidates that violate the action requirement.
- **Sorrifier:** patches invalid candidates with `sorry` only for partial score estimation.
- **Search and reward evaluator:** maintains the AND/OR graph, selects open states using a priority queue, computes scores, and exports scored trajectories.

These components implement the formulation from Section 4. The model proposes candidate proof code, Lean checks it, the graph records the resulting state changes, and the reward evaluator stores both successful and failed attempts for later analysis.

Figure 7.1.1 shows how the framework components interact during search. The loop is not a pure pass-or-fail sampler: every candidate is materialized inside a

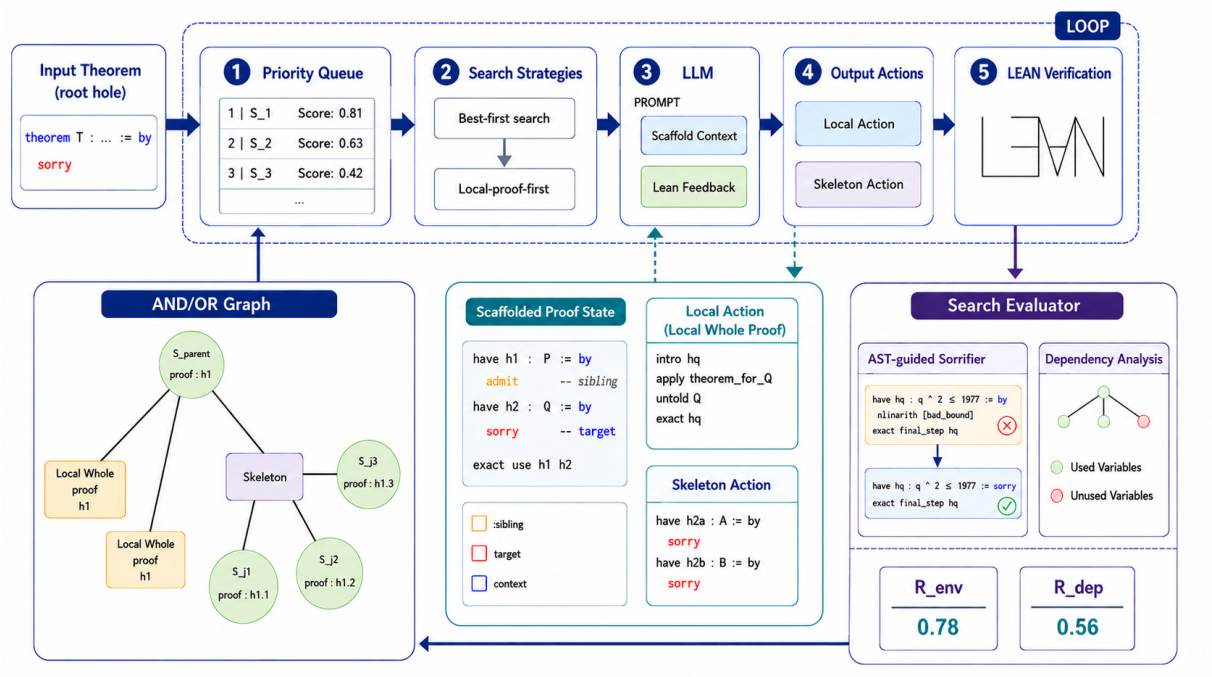

Figure 7.1.1: Overview of the GammaZero framework. The system maintains a priority queue over scaffolded proof states, asks the LLM for local proof or skeleton actions, checks candidates with Lean, applies AST-guided patching when useful, computes syntactic and dependency-based scores, and updates the AND/OR proof-search graph.

scaffold, checked by Lean, and then either accepted as a proof step, patched for partial syntactic signal, or retained as a failed attempt with diagnostic information. The search evaluator combines these signals with graph structure so that the priority queue can focus later model calls on the most promising open proof states.

Figure 7.1.2 connects the abstract search graph to the concrete Lean code checked during verification. A skeleton action creates an AND-node whose children are OR-nodes corresponding to named subgoals. Each child is then solved inside a scaffolded proof context that keeps the surrounding proof structure visible, including sibling declarations and unresolved holes. This allows GammaZero to solve a local subgoal while preserving enough context to stitch the completed child proof back into the root theorem.

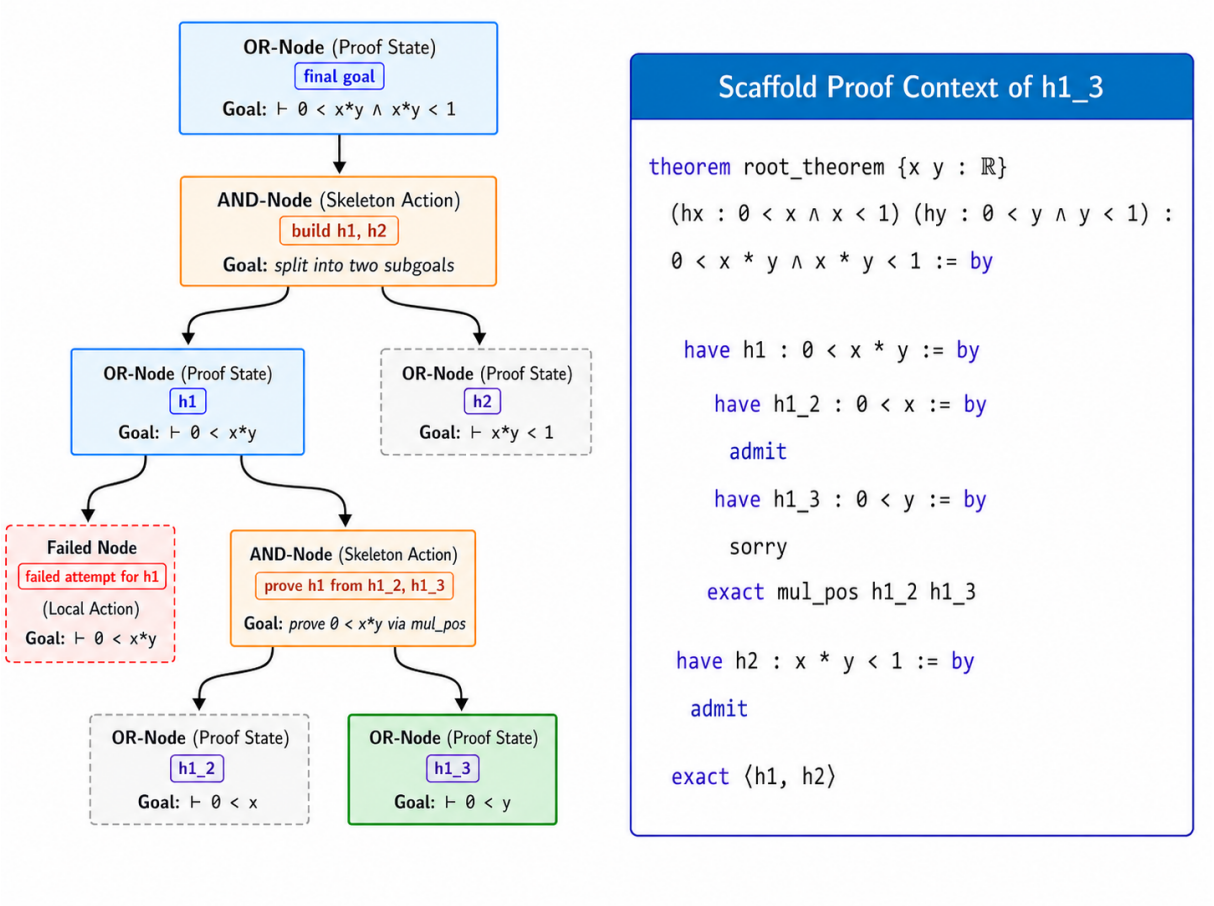

Figure 7.1.2: Illustration of GammaZero's hierarchical AND/OR proof search. OR-nodes represent Lean proof states, while AND-nodes represent skeleton actions that decompose a goal into named subgoals. The scaffolded proof context on the right shows how a child state is embedded back into the surrounding Lean proof, preserving sibling goals and local dependencies.

7.2 Scaffold Rule in the Implementation

The scaffold rule from Section 4 is enforced at the level of candidate extraction. When a skeleton creates child subgoals, each child is checked inside a scaffold derived from the parent proof. The focused child is represented by the target `sorry`, while sibling children are represented by temporary `admit` placeholders.

The model may see the surrounding scaffold, but the accepted action is only the replacement for the target hole. If the returned code changes a sibling `admit`, removes surrounding declarations, or modifies the parent assembly, the action is rejected or recorded as an extraction failure. The local rule is:

> solve exactly this `sorry`; keep every `admit` unchanged.

Each state with a scaffold records the scaffold hash, target kind, target index, and, when available, the target label inferred from the declaration that contains the target. If the target placeholder appears in `have h_step : P := by sorry`, the target label is `h_step`. These fields are part of state identity, so two equal-looking goals are not conflated when they occupy different scaffold positions.

This representation is also used for nested skeletons. A local proof action must replace only the target placeholder and must not modify sibling `admit` placeholders. A child skeleton may introduce further child subgoals, but the extracted replacement is still checked against the original scaffold before it is accepted.

Target labels and target indices make this check stable. A label helps humans read the log; the index helps the system identify the exact placeholder when several similar declarations appear in a single proof body. Together they make the extracted action robust to superficial formatting changes.

7.3 Model Interface and Output Parsing

The model is used as a fixed proposal mechanism. For local proof requests, the prompt asks for Lean proof code that closes the current target and forbids `sorry` and `admit`. For skeleton requests, the prompt asks for a proof outline with child subgoals. `admit` is forbidden, and `sorry` is allowed only for those child subgoals, for example:

```
have h_step : P := by
  sorry
```

The prompt may include optional natural-language reasoning, but only the Lean code block is passed to Lean. The parser extracts the first valid Lean code block, stops at the configured generation limit, and applies action-specific checks before verification. For subgoal scaffolds, the parser compares the returned full scaffold with the expected scaffold and extracts only the replacement for the target `sorry`.

7.4 Lean Feedback and Critique-Correction Loop

GammaZero does not treat every failed generation as an isolated sample. When Lean rejects a local proof action or skeleton action, the system records the failed candidate together with the Lean diagnostics reported for that check. If the same proof state receives another model attempt, the prompt builder can include a short history of recent failures for that state. The next request therefore contains not only the goal and scaffold, but also evidence about what has already failed.

This feedback loop is used as a retry mechanism, not as a proof rule. The model is asked to produce a new action for the same target while avoiding the failed attempt. Lean diagnostics such as unknown identifiers, type mismatches, unsolved goals, and failed tactic applications help focus the retry on the actual reason the previous candidate was rejected. For skeleton actions, the same idea applies at the outline level: a rejected skeleton can be followed by another skeleton request that sees the previous failed outline and Lean's response.

To keep the prompt bounded, GammaZero uses a sliding window over failed attempts rather than storing the full history of a state. In the implementation, only the three most recent failure records are included in the retry context. This keeps the model focused on recent compiler feedback and prevents old mistakes from dominating the context window. The search graph still records the full action history for analysis, but the model prompt receives only the bounded feedback window.

The loop is therefore a compiler-feedback mechanism inside search. It can help the model correct local mistakes across retries, but it does not weaken the verification requirement. A corrected candidate must still be inserted into the scaffold and checked by Lean before it can solve a state or create valid child proof states.

7.5 Lean Verification Interface

Candidates are executed by persistent Lean workers. The returned result includes diagnostics, warnings, placeholder locations, generated proof states, and enough source-position metadata for patching and subgoal extraction. A local proof action succeeds only if Lean closes the current state with no real `sorry` or `admit`. A

skeleton is accepted as a candidate when it passes the skeleton format checks and either closes the current target directly or exposes valid child proof states.

Verification is iterative at the action level. For each generated candidate, GammaZero first materializes the candidate inside the current scaffold: a local proof action is inserted as the replacement for the target `sorry`, while a skeleton is inserted as the parent proof body with child holes temporarily represented by `sorry`. Lean then checks the resulting scaffold, not only the isolated fragment returned by the model. If the candidate is rejected or can only be used for syntactic scoring, the search samples or patches another candidate and repeats this materialize-and-check loop until it finds an accepted action or reaches the configured action and verification limits.

Failed candidates may be sent to the Sorrifier for syntactic score estimation. Patching does not turn a failed local proof action into a solved state, and it does not turn a placeholder-containing skeleton into a final proof. System failures, worker crashes, and timeouts are recorded as failed actions with metadata, not patched as proof fragments.

7.6 Sorrifier and AST-Guided Patching

The Sorrifier is used when a candidate contains useful Lean structure but fails local verification. Rather than discarding the whole generated block, GammaZero asks Lean for syntax information around the diagnostic locations and replaces failing subterms, tactics, or proof blocks with controlled `sorry` placeholders. The patched program is not accepted as a proof. It is used only to measure how much of the candidate survives elaboration and to produce a syntactic reward signal for search and trajectory export.

Figure 7.6.1 illustrates why patching is structural rather than purely textual. The failing declarations are located through Lean's parsed syntax tree, so GammaZero can preserve valid surrounding declarations and patch the smallest useful proof regions. This makes the syntactic reward less brittle: a long candidate with two invalid proof blocks can still receive credit for the code that remains meaningful after those blocks are replaced.

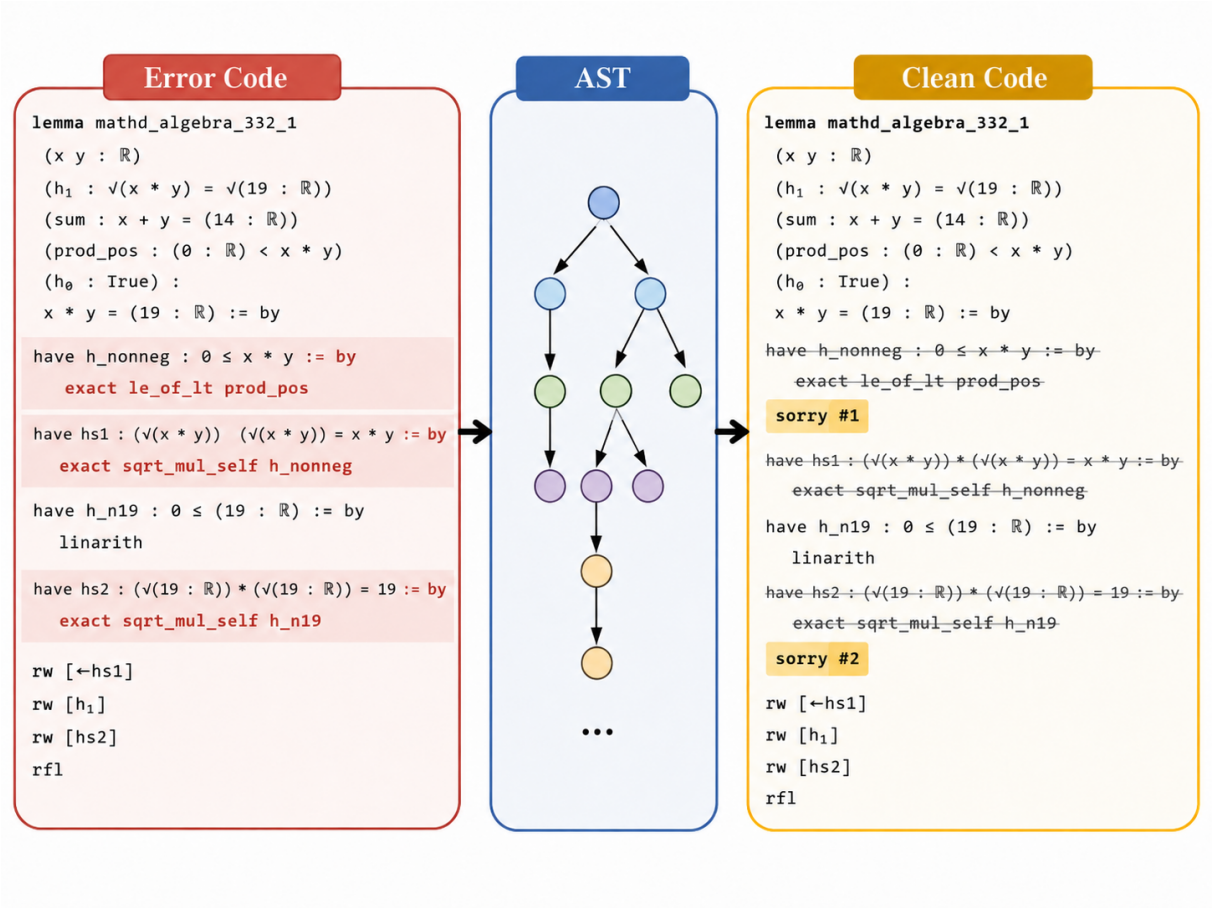

Figure 7.6.1: AST-guided Sorrifier patching. Lean diagnostics identify failing regions in the generated proof, the AST server maps those diagnostics to structured syntax nodes, and the Sorrifier replaces only the invalid regions with `sorry` placeholders. The resulting patched code is used for partial scoring, not as a final proof.

7.7 Lean Metaprogramming Servers

Lean verification is the main computational cost in GammaZero. The implementation therefore avoids starting a fresh Lean process for every small query. Instead, it uses persistent Lean-side services built with Lean metaprogramming. These services expose the information needed by search while reusing the same loaded environment, imports, and worker state across many candidate checks.

The AST server is used for syntax-level operations. Given generated proof code and a diagnostic span, it returns the surrounding syntax nodes and source ranges needed by the Sorrifier. This makes patching structural: the system can patch a tactic, expression, or proof block instead of guessing from raw text. Because

the syntax tree is produced by Lean's own parser, the search does not need to repeatedly reconstruct approximate parse information outside Lean.

The expression server is used after elaboration. It exports selected Lean expressions, including proof bodies and local declarations, into a JSON form that the dependency analyzer can traverse. This supports the r_{dep} reward without asking Lean to re-elaborate the same proof many times for each dependency question. The server also keeps the boundary clear: Lean produces authoritative elaborated expressions, while the external analyzer only reads those expressions and computes usage information.

Together, the persistent workers, AST server, and expression server are resource optimizations, not new logical rules. They reduce process startup cost, repeated imports, duplicated parsing, and repeated elaboration work. Correctness still comes from ordinary Lean verification of the materialized scaffold and, at the end, the reconstructed theorem without placeholders.

7.8 Priority-Queue Search

GammaZero performs best-first selection using a priority queue over open proof states. A run has explicit resource limits, including a maximum number of generated actions, maximum depth, maximum local proof attempts per state, maximum skeleton attempts per state, a global open-state limit, per-depth open-state limits, Lean timeouts, and model generation limits.

Algorithm 1: Priority-Queue Hierarchical Search

1. Initialize the root OR node from the theorem scaffold and insert it into the priority queue.
2. Pop a high-priority open state.
3. Try local proof candidates first, subject to the per-state and global action limits.
4. Verify or repair candidates, record rewards, and mark the state solved if a local proof action closes it without placeholders.
5. If local proof attempts are insufficient or unsuccessful, decide whether the state should request skeletons.

6. Generate, verify, repair, and score skeleton candidates.
7. Commit at most one active skeleton for the parent state; store other promising skeletons as reserved skeletons.
8. Insert child states of the active skeleton into the priority queue.
9. Prune low-priority open states when global or per-depth queue limits are exceeded.
10. Stop when the root is solved or the generated-action, depth, open-state, timeout, or verification-call limits are reached.
11. Run dependency analysis, final stitching, and bottom-up AND/OR reward backup.

Micro-batching is used only as an execution schedule for model calls and Lean verification. It does not define the logical order of expansion. The logical search order is determined by priority scores over open states.

7.9 Local-Proof-First Expansion

The search does not request skeletons immediately for every state. For a new state, GammaZero first samples local proof actions because many Lean goals can be closed inside the current scaffold and should not be decomposed. A skeleton request is made only after local proof attempts fail to solve the state, after the state has used enough local attempts, or after priority signals indicate that decomposition may be useful.

The rule is adaptive. If failed local proof attempts have high r_{env}, the system may continue local proof sampling longer because the model is producing Lean code with high syntactic reward. If local proof attempts have low syntactic reward or repeatedly leave the same state open, the state becomes eligible for skeleton generation earlier.

7.10 Skeleton Commitment and Fallback

Several skeletons can be generated for the same proof state, but they may organize the proof around different proof structures. Solving children from one skeleton does

not necessarily help complete another skeleton. GammaZero therefore commits to one active skeleton per parent state, while keeping other promising skeletons as reserves. The committed skeleton receives focused search effort: its children enter the priority queue, and the parent does not keep requesting unrelated skeletons while that skeleton remains active.

Other promising skeletons are stored as reserved skeletons. If the committed skeleton fails, reaches its search limits without progress, or becomes stale after repeated rounds, a reserved skeleton can be activated. Duplicate skeletons are rejected before reservation when they expose the same scaffold structure and equivalent child subgoals. This keeps search effort concentrated without making the first skeleton choice irreversible.

7.11 Priority Scores

The priority scores are heuristics for allocating limited search effort. They are not intended to measure mathematical truth or proof elegance. Their role is to decide which open state should receive the next model and Lean calls. The score estimates search promise, not theorem truth.

The state priority score combines signals that estimate whether expanding a state is likely to finish an active proof:

$$\begin{aligned} Q_{\text{state}}(s) = {} & w_I I(s) + w_R R(s) + w_B B(s) \\ & - w_d d(s) - w_T T(s) - w_S S(s) - w_U U(s). \end{aligned}$$

Term	Meaning
s	Open OR-node, i.e., a proof state waiting for expansion.
$I(s)$	Incoming priority inherited from the skeleton that created s.
$R(s)$	Best syntactic reward obtained by a local proof candidate tried on s.
$B(s)$	Progress bonus for children of the committed skeleton, especially when a child would complete the parent.
$d(s)$	Search depth of s.
$T(s)$	Number of local proof attempts made directly at s.
$S(s)$	Number of skeleton-generation attempts made directly from s; descendant skeletons are not counted.
$U(s)$	Number of search rounds where the committed skeleton for s is stale, low-progress, or keeps child states open despite additional effort.
$w_\cdot$	Non-negative heuristic weights controlling queue-order sensitivity.

Incoming skeleton score transfers priority from a promising decomposition to its children. The committed-skeleton progress bonus favors children of the active skeleton, especially the last open child. Depth and retry penalties reduce priority for states that are deep, repeatedly attempted, or attached to skeletons with little progress. In particular, $U(s)$ penalizes budget spent inside an unproductive committed decomposition even when no new skeleton proposal is generated.

Skeletons are scored before commitment or reservation:

$$\begin{aligned} Q_{\text{skel}}(a) = {} & u_E E(a) + u_P P(a) + u_C C(a) \\ & - u_d d(\text{parent}(a)) - u_\rho \rho(a). \end{aligned}$$

Term	Meaning
a	Candidate skeleton action proposed for a parent proof state.
$E(a)$	Syntactic reward measuring how much of the generated skeleton survives Lean-guided patching.
$P(a)$	Priority propagated from the parent OR-node into the candidate skeleton.
$C(a)$	Child-count score: rewards useful decomposition but penalizes too many branches.
$d(\text{parent}(a))$	Search depth of the parent state that proposed a.
$\rho(a)$	Repair indicator, equal to 1 if the skeleton required patching and 0 otherwise.
$u.$	Non-negative heuristic weights for skeleton ranking.

This score prefers skeletons that are clean, have high syntactic reward, and are proposed from promising parent states. A repaired skeleton can still be useful, but a clean skeleton is preferred when other signals are similar.

In practice, the score must interact with the search budget. A skeleton that is only slightly worse than the best one may still be worth reserving if the parent state is important and the remaining queue is small. Conversely, a very poor skeleton should not remain in the queue just because it is technically valid. The scoring policy is therefore not a universal ranking of mathematical quality; it is a local decision rule for allocating limited verification effort.

The formula is intentionally heuristic rather than normative. Its purpose is to prefer states that are close to completing the currently committed proof structure, not necessarily states that are globally shortest or most elegant. A deep state with one nearly solved child may outrank a shallow state with no clear proof direction if the committed skeleton is near completion. Conversely, a deep state with repeated failed local proof attempts and stale skeletons should drift downward in the queue even if it has not yet exceeded its hard search limits. This is the practical difference between best-first search and a depth-ordered rollout: the former tries to spend effort where it is most likely to close something useful, while the latter merely walks level by level.

Queue pruning is equally important. A proof search graph can accumulate many

open states that are no longer promising because they are dominated by a better sibling, belong to a stale skeleton, or have already consumed too many attempts. If the queue grows without pruning, the system wastes verification time on states that are unlikely to contribute to the final proof. GammaZero therefore treats pruning as part of the search policy, not as an afterthought. The pruning count is one of the recorded metrics because it measures how aggressively the search focused on promising open states.

Skeleton scoring plays a similar role at the decomposition level. A skeleton that creates a compact parent proof with high syntactic reward may be activated immediately, while a skeleton that repeatedly repairs into a fragile structure may remain reserved or be discarded. The score does not claim to predict theorem truth; it only predicts whether the candidate is likely to be useful inside the current proof graph. This separation between utility and truth is what allows GammaZero to keep using partially successful actions as data even when the final theorem is not solved.

7.12 Final Proof Construction

When child states are solved, their proof bodies are stitched into the corresponding child subgoals in the parent skeleton. This process propagates upward: solved children close a skeleton, a solved skeleton closes its parent state, and the root theorem is reconstructed from the selected action chain.

After stitching, dependency analysis may identify declarations that are solved but unused by the final proof expression. Such unused declarations may be removed, and Lean is asked to verify the cleaned theorem again. The accepted final proof must contain no real `sorry` or `admit`; placeholder-compilable scripts are retained only as scored training records.

This final verification step deserves emphasis because it closes the loop between search and proof correctness. The search graph may contain many branches, repaired candidates, reserved skeletons, and unused declarations. None of those artifacts should leak into the accepted theorem. The final checker therefore acts on the reconstructed proof body, not on an abstract summary of the search. If the cleaned proof no longer verifies, the theorem is not considered solved. In other words, the search record may be richer than the final proof, but the final proof

remains the only object that matters for correctness.

7.13 End-to-End Lifecycle

With the implementation pieces in place, the full run can be summarized as follows. GammaZero starts from a theorem whose proof body contains one root placeholder. Lean returns the initial proof state, and the state is inserted into the priority queue. The search then selects an open state, serializes its scaffold, local context, and target goal, and asks the model for local proof attempts.

Each candidate is stitched into the current scaffold and checked by Lean. If a local proof action closes the target without placeholders, the state becomes solved and the solution can propagate upward. If local proof attempts fail but still receive high syntactic scores, the state may receive more local proof attempts. If local closure appears unlikely, the state becomes eligible for skeleton generation.

For a skeleton, the parent proof body is checked with temporary child holes. If Lean accepts the structure, GammaZero extracts each child hole as a new proof state with its own scaffold. The best skeleton for the parent is committed, its child states are inserted into the priority queue, and other promising skeletons may be kept as reserves. Search then continues from the highest-priority open child, which may itself be solved locally or decomposed by another skeleton.

When a child is solved, its proof body is inserted into the corresponding hole of its parent skeleton. If all children of a committed skeleton are solved, the skeleton closes its parent state. This propagation can continue until the root theorem is solved. The final theorem is reconstructed from the selected solved actions, optionally cleaned by dependency analysis, and verified again by Lean. The exported search record may contain failed actions, repaired candidates, reserved skeletons, and pruned states, but the final proof contains only the selected verified path.

Chapter 8

Experiments

8.1 Experimental Scope

The goal of the experiments is to evaluate GammaZero as an implemented proof-search and data synthesis framework. The primary question is whether the system can run end-to-end: generate candidate actions, verify them with Lean, expand the subgoals introduced by skeleton actions, patch invalid generations, score partial proof signal, and finally produce Lean-verified proofs without placeholders when the search succeeds.

The main correctness metric is root solved rate without `sorry` or `admit`. However, root solved rate alone does not explain how the system behaves. Since GammaZero is designed to produce structured search trajectories for future policy optimization, the experiments also report diagnostic metrics about search effort, patching, and scoring.

The evaluation therefore focuses on three aspects:

- **Proof success:** whether the final reconstructed proof is accepted by Lean without placeholders.
- **Search effort:** how many actions are generated, how many Lean verification calls are used, how deep the solved proof becomes, and how often the queue is pruned.
- **Patching and scoring signals:** how often invalid generations are patched

by the Sorrifier and how much score their surviving code receives.

These measurements are intended to describe the behavior of the implemented framework rather than to claim that every design choice is globally optimal. For example, comparing priority-queue search with a simpler expansion schedule can show whether the search order matters in this implementation, but it is not a universal theorem about best-first search.

The resulting search records are treated as scored proof-search trajectories. They are suitable for future policy optimization, but this thesis reports only the search, patching, scoring, and final verification results obtained from the current implementation.

8.2 Setup

Lean environment. All experiments are conducted in Lean 4 using version `v4.29.0-rc1`. The project uses Mathlib [13] from the `leanprover-community/mathlib4` repository, pinned to revision:

```
33a7291a345d718bca41f8ae8a9bae365d8f2a3e
```

The Mathlib input revision is `v4.29.0-rc1`. This pinned environment ensures that all Lean executions, diagnostics, and proof-state reports are evaluated under a fixed library version.

Hardware. Experiments are run on a Linux workstation with an NVIDIA RTX 3090 GPU with 24GB VRAM, 64GB system RAM, and an Intel Xeon Silver 4114 CPU. The CPU has 20 logical threads, with 10 physical cores and 2 threads per core. Lean execution is parallelized using multiple Lean workers.

Policy model. Candidate local proof actions and skeleton actions are generated using `Gemini 3 Flash Preview`. The model is used as a fixed proposal mechanism for search-data generation; no additional fine-tuning or reinforcement learning update is performed in this thesis. The model samples Lean code conditioned on

serialized proof states and task-specific prompts for local proof or skeleton action generation.

Search configuration. For each selected proof state, the search samples candidate actions subject to global and per-state generation limits. Local proof attempts are tried before skeleton requests, and the priority queue controls which open state is expanded next. The maximum search depth is set to $D = 10$, and the maximum number of graph nodes per theorem is set to $N = 512$.

Lean execution uses 4 parallel workers. Each Lean call has a timeout of 120 seconds, and the Lean heartbeat limit is set to 100000. The Sorrifier is allowed up to 50 patching cycles for a failed generation.

Generation configuration. The maximum generation length is set to 8192 new tokens. Sampling uses temperature 1.0, with the default nucleus-sampling setting of the backend API, approximately $p = 0.95$ when applicable. The generated output may include a `<think>` block, but only the extracted Lean code block is submitted to Lean for execution.

8.3 Dataset

The evaluation uses the validation and test splits of miniF2F-v2, introduced by Ospanov et al. [11] as a corrected version of the original miniF2F benchmark [17]. The original miniF2F benchmark contains 488 formalized olympiad-level mathematics problems and has become a standard benchmark for neural theorem proving. However, the revisited study identifies mismatches and errors between informal and formal statements in the original benchmark and provides corrected versions with verified alignment between the informal problem statements and the corresponding Lean formalizations.

In this thesis, we evaluate on the `miniF2F-v2s` validation and test splits, the simplified aligned version of miniF2F-v2. The dataset is suitable for testing both local tactic generation and higher-level proof decomposition because the problems cover competition-style mathematics such as algebra, number theory, and inequalities.

8.4 Metrics

We report metrics in three groups: proof success, search cost, and patching and scoring signals.

Proof success. The main correctness metric is root solved rate without placeholders. A root theorem is counted as solved only when the final reconstructed proof is accepted by Lean and contains no real `sorry` or `admit`. This metric is deliberately strict: placeholder-compilable proofs, partially solved skeletons, and patched scripts do not count as solved roots.

For external context, we compare against the strongest whole-proof prover reported by Ospanov et al. [11] on miniF2F-v2s. This comparison is not a strict same-budget comparison, because their results use a different model, prover architecture, and evaluation setup. It is included as a real published reference point rather than as an internal ablation.

Search cost. Generated actions count all local proof and skeleton proposals produced by the model interface. Lean verification calls count executions sent to Lean, including calls used during patching and final verification. These two numbers are reported separately because one generated action may trigger several Lean calls if it requires patching, while an action rejected by a format check may trigger no full verification.

We also report average solved depth. A depth of zero means that the theorem was solved directly at the root. A larger depth means that the proof used nested decomposition through skeleton actions.

Patching and scoring signals. Patched candidates measure how often the Sorrifier is used. A high patching rate can have two meanings: the model may be producing many invalid candidates, or the patching step may be preserving useful code that would otherwise be discarded. For this reason, patching rate is interpreted together with the syntactic reward r_{env} and the dependency reward r_{dep}.

8.5 Main Results

Table 8.5.1 compares GammaZero with two Lean 4.29 baselines built around the Kimina prover: direct proof sampling and the APOLLO recursive-repair pipeline [10]. All three methods are reported on the same 87 evaluated test problems completed in this thesis. The comparison should therefore be read as an architectural reference point rather than a controlled same-model, same-budget benchmark.

Method	Lean Env.	Generation Setting	Problems	Solve Rate
Kimina-Prover-Preview-Distill-7B	Lean 4.29	pass@32	87	53.28%
APOLLO + Kimina-Prover-Preview-Distill-7B [10]	Lean 4.29	pass@32, recursive depth = 2	87	53.69%
GammaZero	Lean 4.29	max graph nodes = 512	**87**	**67.82%**

Table 8.5.1: Main proof-success comparison under Lean 4.29-style environments on the same 87 evaluated test problems. The Kimina and APOLLO baselines use direct sampling and recursive repair, respectively; GammaZero uses bounded hierarchical graph search.

The direct Kimina baseline samples 32 proof attempts for each theorem.

APOLLO extends the same model with recursive proof recovery and placeholder-guided repair, using a maximum recursive depth of 2. Both baselines use the same Kimina prover model, with temperature 0.6, top-p 0.95, and maximum generation length 16384.

GammaZero is evaluated under the same Lean 4.29-style environment but uses a different search paradigm based on hierarchical AND/OR search, scaffold-aware subgoal solving, and dependency-aware reward-guided exploration. Unlike direct-sampling baselines, GammaZero allocates search effort dynamically across local proof actions and skeleton decompositions under a bounded graph-search budget. It solves 59 out of 87 evaluated test theorems, for a solved rate of 67.82%. The result should not be overread as a strict same-model comparison, but it shows that the hierarchical search design can substantially increase proof coverage on the evaluated test set while preserving placeholder-free final verification.

All 59 solved roots pass final verification after proof stitching, so the reported successes are placeholder-free Lean proofs. The search graph may contain patched

Evaluation set	Theorems	Avg. Actions	Avg. Lean Calls	Patching	Avg. Input Tok.	Avg. Output Tok.
miniF2F-valid	33	124.1	229.7	77.2%	424.2K	172.7K
miniF2F-test	87	162.5	294.5	79.8%	401.1K	208.3K

Table 8.5.2: Search cost and patching summary. Actions, Lean calls, and token counts are reported as averages per theorem; the theorem column gives the number of evaluated problems in each split.

candidates, temporary placeholders, failed branches, and reserved skeletons, but a root is counted as solved only after the selected proof path is reconstructed and checked again by Lean.

8.6 Ablation Study

To isolate the effect of hierarchical expansion, we compare GammaZero with a root-only local proof variant on the same evaluated test set. This ablation uses the same root local proof budget and the same Lean feedback retry mechanism, but disables skeleton expansion and deeper graph search. It is therefore an internal ablation, not an external theorem-proving baseline.

The root-only variant starts from the same initial theorem scaffold as GammaZero, with one root `sorry` hole. It repeatedly asks the model for local proof bodies that replace only this root hole, materializes each candidate in the theorem, and checks the result with Lean. Failed attempts may still feed the same critique-correction prompt window used by the full system, so the comparison does not remove compiler feedback. What it removes is hierarchy: the variant never accepts skeleton actions, never creates child proof states, and never searches below the root. A theorem is solved by this variant only if one of the root-level local proof attempts closes the whole theorem directly without `sorry` or `admit`.

Method	Solved	Solve Rate	Gain
Root-only local proof	41 / 87	47.13%	–
GammaZero	**59 / 87**	**67.82%**	**+20.69%**

Table 8.6.1: Root-only ablation. Gain is the absolute improvement from enabling skeleton expansion and deeper graph search under the same test set and search budget.

The ablation shows that root-level local proof attempts solve a substantial fraction of the evaluated theorems. However, enabling skeleton expansion adds 18 solved theorems beyond the 41 root-level direct solves. This suggests that the hierarchical part of GammaZero is not merely diagnostic machinery: it contributes additional solved roots beyond direct completion at the initial proof state.

We further separate the solved theorems according to the depth of the final accepted proof path. A direct root solve has solved depth 0: the theorem is closed at the root node by local proof actions without committing to a skeleton decomposition. A hierarchical solve has solved depth at least 1: the final proof uses at least one skeleton-created child state in the committed AND/OR proof tree.

Solve category	Solved / Total	Rate
Direct root solves (depth 0)	41 / 87	47.13%
Hierarchical skeleton solves (depth ≥ 1)	18 / 87	20.69%
Total solved	**59 / 87**	**67.82%**

Table 8.6.2: Depth-based decomposition of solved theorems on the evaluated test set. Direct root solves correspond to flat local proof generation; hierarchical skeleton solves are additional successes whose final proof path uses at least one skeleton decomposition.

This depth split is the most direct internal measurement of the contribution of hierarchy. The 41 direct root solves estimate the effective flat tactic baseline for the proposal model under the same root generation and verification conditions. The 18 additional hierarchical solves correspond to a 20.69 percentage-point absolute gain from enabling skeleton decomposition and AND/OR graph search.

8.7 Patching and Scoring Signals

Across the 33 validation rollout logs, the Sorrifier patches 3,161 out of 4,095 generated actions, giving a patching rate of 77.19%. On the evaluated 87-theorem test set, GammaZero issues 2,216 skeleton requests, of which 1,768 require patching after a failed Lean check. This gives a patching rate of 79.8%. These high patching rates show that invalid Lean generations are common in this setting. They also show why binary success or failure is not sufficient for data generation: many

Theorem	Nodes	Skeletons	Output Tok.
aime_1983_p9	65	4	22,393
aime_1988_p3	272	18	86,489
aime_1990_p2	158	8	35,680
aime_1996_p5	427	50	285,986
algebra_amgm_faxinrrp2msqrt2geq2mxm1div2x	62	4	12,366
algebra_amgm_prod1toneq1_sum1tongeqn	108	8	31,797
amc12_2000_p15	58	4	16,232
amc12a_2003_p24	236	20	72,798
amc12a_2008_p15	316	37	81,946
amc12a_2008_p8	64	4	11,011
amc12a_2010_p11	85	8	28,078
amc12a_2011_p18	126	8	47,663
amc12b_2004_p3	55	4	15,947
imo_1961_p1	76	4	24,878
imo_1964_p1_1	194	14	50,416
imo_1964_p1_2	70	6	17,511
imo_1965_p1	470	60	272,345
imo_1966_p4	541	46	218,159

Table 8.6.3: The 18 test theorems solved by hierarchical skeleton search. These theorems were not closed by direct root solving; their final accepted proofs required skeleton-created child states.

invalid candidates still contain locally meaningful structure that can be patched and scored.

The low average dependency reward r_{dep} should be read carefully. Most generated actions do not produce a complete dependency structure: they may fail, be patched with placeholders, or leave the main goal unsolved. Such actions naturally receive little or no dependency credit. A smaller number of useful actions receive high dependency scores and are the actions that matter for successful stitched proofs.

This is why r_{env} and r_{dep} are reported together. The syntactic reward measures how much generated Lean text survives patching. The dependency reward is sparser: it gives credit only when generated declarations are both solved and used by the checked proof. The average gap therefore reflects the sparsity of genuinely useful proof structure, not a claim that every patched fragment is semantically valuable.

A detailed trajectory case study for `aime_1991_p9` is included in Appendix .1.

8.8 Discussion

The main result shows that GammaZero can solve a large fraction of the evaluated theorem instances while preserving a strict correctness boundary: final success is counted only after Lean verifies the reconstructed proof without placeholders. The final stitching success count is also important. It indicates that the graph can contain many partial or failed branches while still producing a clean final proof when a complete path is found.

The patching and scoring results show that invalid generations are not rare. Most generated actions require some form of patching, but many still preserve enough local structure to receive useful scores. This supports the design decision to separate proof correctness from trajectory scoring. A patched script is not a proof, but it can still help describe how much of the generated Lean code was locally meaningful.

The root-only ablation shows that local proof generation is already strong, but not sufficient for the evaluated subset. Enabling skeleton expansion and deeper graph search adds solved theorems beyond direct root completion, which supports the central design choice of combining local proof actions with hierarchical search.

The gap between r_{env} and r_{dep} is one of the most important findings. It shows that local syntactic survival and final proof contribution are different properties. A generated fragment may survive patching but still not matter to the completed proof. This justifies using expression-tree dependency analysis as a separate structural signal.

At the same time, the results should not be overinterpreted. The external comparison uses published systems with different models and budgets, while GammaZero is reported under the implementation constraints of this thesis. The evaluation also uses a fixed proposal model, a fixed Lean environment, and a finite search budget. Unsolved theorems are not mathematically impossible; they are unsolved under the current configuration.

8.9 Reproducibility Considerations

Reproducibility is difficult in LLM-guided theorem proving because several components can change independently. The model backend may update, sampling can be stochastic, Lean and Mathlib versions can change theorem names or elaboration behavior, and worker timeouts can depend on hardware load. For this reason, the experimental setup records the Lean version, Mathlib revision, model name, generation parameters, worker count, timeout, search limits, and dataset split. These details are not administrative noise; they define the environment in which a proof attempt is meaningful.

The most important reproducibility boundary is the Lean environment. A proof that verifies under one Mathlib revision may fail under another because a lemma was renamed, a simp rule changed, or an instance search path became different. The thesis therefore treats the pinned Lean and Mathlib versions as part of the experimental artifact. Search records should be interpreted relative to that environment, and future comparisons should either use the same environment or explicitly report the migration.

LLM sampling introduces a second source of variance. Even with fixed prompts and temperature, hosted model APIs may not provide perfect determinism. The thesis therefore emphasizes aggregate search metadata and trajectory structure in addition to root solve rate. If one run solves a theorem and another does not, the difference may come from sampling rather than from the architecture. Recording generated actions, priority scores, patching results, and final proof paths makes it possible to inspect why a run succeeded or failed.

8.10 Threats to Validity

There are four main threats to validity. The first is benchmark validity. miniF2F-style datasets are useful because they provide formal theorem statements, but they do not cover all styles of mathematics or all Lean proof patterns. A system that works on olympiad algebra may behave differently on topology, category theory, or large-library engineering proofs. This thesis therefore treats miniF2F-v2s as a controlled evaluation setting rather than as a complete measure of theorem-proving ability.

The second threat is model dependence. GammaZero is designed to be model-agnostic, but the quality of generated local proof actions and skeleton actions strongly affects the observed search behavior. A stronger model might need fewer patches and produce better skeletons; a weaker model might make the search appear worse than the architecture itself. The experiments should therefore report the proposal model clearly and avoid claiming that results transfer automatically to all LLMs.

The third threat is heuristic dependence. Priority scores, skeleton commitment thresholds, and stale skeleton rules are hand-designed in the current implementation. They are intended to be reasonable engineering choices, not optimal search theory. The present experiments do not isolate each heuristic component, so the exact numbers should be interpreted as evidence about this implementation rather than universal constants. The root-only ablation isolates the broad contribution of hierarchical expansion, but finer-grained ablations of the individual heuristic terms are left for future work.

The fourth threat is reward interpretation. Dense reward can make partial progress visible, but it can also overemphasize syntactic survival if not paired with structural checks. GammaZero mitigates this by combining line-based patching scores with dependency analysis and final Lean verification. Nevertheless, the reward values should be understood as training signals, not as mathematical measures of elegance or insight.

Chapter 9

Conclusion and Limitations

This thesis presents a hierarchical framework for Lean 4 theorem proving that separates local proof resolution from skeleton decomposition. The design addresses sparse reward through AST-guided recovery and handles decomposition credit through dependency-aware AND/OR backup. The key correctness boundary is that patched scripts with `sorry` are used only for learning signal; final success requires a fully verified Lean proof without placeholders.

The current design still has limitations. Dependency-based credit assignment reflects the proof that was found, not necessarily the shortest or mathematically minimal proof. Open subgoals within finite search limits cannot always be distinguished from genuinely impossible subgoals. The effectiveness of skeleton generation depends on the policy model's ability to propose useful subgoals. These limitations should be evaluated empirically in the experiments.

There are also broader limitations that are easier to state than to eliminate. The model interface assumes that the LLM can produce output that is at least parseable into Lean code blocks. If the model drifts into natural language, malformed markdown, or repetitive syntax errors, the Sorrifier can recover only a fraction of the damage. The scaffold-aware rule assumes that subgoals can be cleanly associated with a single target hole, which is true for the proof bodies GammaZero is designed to handle but may be awkward for very irregular generated code. The priority queue assumes that the heuristic score is at least informative enough to rank open states roughly by promise; if that score is poor, the system can still search, but it will search more expensively.

The dependency reward also has an important interpretive limit. It measures usage in the elaborated proof that was found, not a global theorem-theoretic notion of necessity. A declaration can be unused in the final stitched proof and still have been useful as an intermediate exploration step. Likewise, an unresolved child may block the final theorem even if it looks locally reasonable. These are not bugs in the reward definition; they are reminders that the thesis is about search data, not formal proof minimization.

Future work naturally follows from this framing. The collected search records could be used for policy fine-tuning or GRPO-style optimization. Retrieval could be integrated so that skeleton prompts and local proof prompts receive relevant premises automatically. Larger or more diverse theorem libraries could be added so that the search graph contains a broader variety of decomposition patterns. The skeleton scorer could also be replaced by a learned ranking model once enough trajectories have been collected. Each of these directions would build on the current thesis, but none of them is required for the present contribution: a verifiable, scaffold-aware, priority-queue proof-search system that produces structured reward data.

Another direction is to improve the model interface itself. The current separation between local proof and skeleton prompts is simple and inspectable, and the implementation already feeds recent Lean failures back into retry prompts. A future system could add richer search-history summaries, dependency feedback, or retrieved examples of similar proof patterns. A model might learn that certain goals should be attacked directly while others should be decomposed. It might also learn to propose smaller skeletons that introduce only subgoals likely to be solved by the current local proof policy. Such improvements would turn the current hand-designed local-proof-first rule into a learned decision about when decomposition is useful.

The final long-term goal is a closed loop: generate search trajectories, score them with Lean-backed signals, train a better proposal model, and then use the improved model to generate higher-quality trajectories. This thesis implements the first half of that loop. It defines the graph structure, verification requirements, repair mechanism, dependency analysis, and trajectory export needed for such training. The remaining optimization step is left for future work because it requires substantial compute and careful experimental design. Keeping that boundary explicit avoids

overstating the contribution while still showing how the system can support future reinforcement learning.

The thesis also leaves room for better proof-object analysis. The current dependency analyzer focuses on whether generated declarations are used by the elaborated proof expression and whether they contain placeholders. A richer analyzer could measure proof-term size, count references to library lemmas, detect whether a declaration is used only through a trivial wrapper, or compare multiple stitched proofs for the same theorem. Such analysis would help distinguish merely valid proofs from compact or pedagogically useful proofs. That is outside the current scope, but the graph representation makes the extension natural.

Finally, the system could benefit from richer feedback to the model at generation time. The current implementation already feeds recent Lean failures back into retry prompts for the same state. Future prompts could add higher-level summaries of reserved skeletons, dependency feedback from previous decompositions, or retrieved examples from similar theorems. This would give the model more context about the search history while preserving the same strict verification boundary: whatever the model proposes, Lean must still check the final proof.

Bibliography

[1] Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf, Edward W. Ayers, Dragomir Radev, and Jeremy Avigad. ProofNet: Autoformalizing and formally proving undergraduate-level mathematics, 2023.

[2] Kshitij Bansal, Sarah M. Loos, Markus N. Rabe, Christian Szegedy, and Stewart Wilcox. HOList: An environment for machine learning of higher order logic theorem proving. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 454–463. PMLR, 2019. URL `https://arxiv.org/abs/1904.03241`.

[3] Luoxin Chen, Jinming Gu, Liankai Huang, Wenhao Huang, Zhicheng Jiang, Allan Jie, Xiaoran Jin, Xing Jin, Chenggang Li, Kaijing Ma, Cheng Ren, Jiawei Shen, Wenlei Shi, Tong Sun, He Sun, Jiahui Wang, Siran Wang, Zhihong Wang, Chenrui Wei, Shufa Wei, Yonghui Wu, Yuchen Wu, Yihang Xia, Huajian Xin, Fan Yang, Huaiyuan Ying, Hongyi Yuan, Zheng Yuan, Tianyang Zhan, Chi Zhang, Yue Zhang, Ge Zhang, Tianyun Zhao, Jianqiu Zhao, Yichi Zhou, and Thomas Hanwen Zhu. Seed-Prover: Deep and broad reasoning for automated theorem proving, 2025.

[4] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In André Platzer and Geoff Sutcliffe, editors, *Automated Deduction – CADE 28*, pages 625–635, Cham, 2021. Springer International Publishing. doi: 10.1007/978-3-030-79876-5_37.

[5] Kefan Dong, Arvind Mahankali, and Tengyu Ma. Formal theorem proving by rewarding LLMs to decompose proofs hierarchically, 2024.

[6] Albert Q. Jiang, Wenda Li, Szymon Tworkowski, Konrad Czechowski, Tomasz Odrzygóźdź, Piotr Miłoś, Yuhuai Wu, and Mateja Jamnik. Thor: Wielding hammers to integrate language models and automated theorem provers, 2022.

[7] Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs, 2022.

[8] Cezary Kaliszyk, Josef Urban, Henryk Michalewski, and Mirek Olsák. Reinforcement learning of theorem proving, 2018.

[9] Guillaume Lample, Marie-Anne Lachaux, Thibaut Lavril, Xavier Martinet, Amaury Hayat, Gabriel Ebner, Aurélien Rodriguez, and Timothée Lacroix. HyperTree proof search for neural theorem proving, 2022.

[10] Azim Ospanov, Farzan Farnia, and Roozbeh Yousefzadeh. APOLLO: Automated LLM and Lean collaboration for advanced formal reasoning, 2025.

[11] Azim Ospanov, Farzan Farnia, and Roozbeh Yousefzadeh. miniF2F-Lean revisited: Reviewing limitations and charting a path forward, 2025.

[12] Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving, 2020.

[13] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, CPP 2020, pages 367–381, New York, NY, USA, 2020. Association for Computing Machinery. doi: 10.1145/3372885.3373824.

[14] Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models, 2022.

[15] Huajian Xin, Z. Z. Ren, Junxiao Song, Zhihong Shao, Wanjia Zhao, Haocheng Wang, Bo Liu, Liyue Zhang, Xuan Lu, Qiushi Du, Wenjun Gao, Qihao Zhu, Dejian Yang, Zhibin Gou, Z. F. Wu, Fuli Luo, and Chong Ruan. DeepSeek-Prover-V1.5: Harnessing proof assistant feedback for reinforcement learning and monte-carlo tree search, 2024.

[16] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems*, 2023. URL `https://arxiv.org/abs/2306.15626`.

[17] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. miniF2F: A cross-system benchmark for formal olympiad-level mathematics. In *International Conference on Learning Representations*, 2022. URL `https://openreview.net/forum?id=9ZPegFuFTFv`.

.1 Trajectory Case Study: `aime_1991_p9`

This appendix examines the complete, successful search trajectory for the theorem `aime_1991_p9` from the `miniF2F-valid` split. Proving this Olympiad-level trigonometry and algebra problem requires establishing:

$$\forall (x : \mathbb{R})(m : \mathbb{Q}), \quad \left(\frac{1}{\cos x} + \tan x = \frac{22}{7}\right) \wedge \left(\frac{1}{\sin x} + \cot x = m\right) \implies m.\text{den} + m.\text{num} = 44$$

where $m \in \mathbb{Q}$ is the rational target, represented as the fraction $m = \frac{29}{15}$, so that $m.\text{num} = 29$ and $m.\text{den} = 15$.

The theorem was successfully solved by GammaZero at a search depth of 7, generating 506 actions, making 916 Lean verification calls, and containing 545 nodes (39 proof-state OR-nodes and 506 action AND-nodes) in the final search graph. Below we trace the exact nested skeleton structure, the coordination of local proofs, and present the complete reconstructed Lean 4 proof.

.1.1 Hierarchical Search Depth-7 Tree Topology

To illustrate how GammaZero's hierarchical search operates, we map the exact path of the committed proof tree. For context, the key mathematical goals associated with each state ID (s_i) in the tree are:

- s_0: Root goal ($\uparrow m.\text{den} + m.\text{num} = 44$).
- s_1: Intermediate half-angle tangent landmark ($\tan(x/2) = 15/29$).
- s_2: Main trigonometric equation transformed to the half-angle parameter.
- s_3, s_{13}, s_{14}: Rational parameterizations of $\cos x$, $\tan x$, and their algebraic combination.
- s_4: Double-angle expansion for $\cos x$.
- s_5, s_6, s_7, s_9: Derivation of the cosine-square substitution using secant and tangent-square identities ($1 + \tan^2(x/2) = 1/\cos^2(x/2)$).
- s_8, s_{11}: Critical non-degeneracy conditions ($\cos^2(x/2) \neq 0$ and $\cos(x/2) \neq 0$) to verify division validity.
- s_{21}: Solving the rational algebraic equation $\frac{1+t}{1-t} = \frac{22}{7} \implies t = \frac{15}{29}$.
- s_{22}, s_{23}, s_{27}, s_{28}, s_{29}, s_{30}, s_{31}: Recursive scaffolding to establish the target cotangent/cosecant identity ($1/\sin x + \cot x = 1/\tan(x/2) = m = 29/15$).

- s_{33}, s_{34}: Final extraction of the numerator (m.num = 29) and denominator (m.den = 15) of the rational result.

The tree below shows the compact logical hierarchy of how the goals are decomposed and solved:

```
D0 s0  final rational-denominator goal
`- a32 skeleton
   |- D1 s1  half-angle tangent value
   |  `- a65 skeleton
   |     |- D2 s2  transform first hypothesis to half-angle form
   |     |  `- a98 skeleton
   |     |     |- D3 s3  cosine half-angle rational form
   |     |     |  `- a133 skeleton
   |     |     |     |- D4 s4  cosine double-angle identity -> a134
   |     |     |     |- D4 s5  cosine-square denominator form
   |     |     |     |  `- a170 skeleton
   |     |     |     |     |- D5 s6  secant-square identity
   |     |     |     |     |  `- a211 skeleton
   |     |     |     |     |     |- D6 s7  tan-square identity
   |     |     |     |     |     |  `- a258 skeleton
   |     |     |     |     |     |     |- D7 s8  nonzero cosine-square -> a265
   |     |     |     |     |     |     `- D7 s9  tangent-square algebra -> a289
   |     |     |     |     |     `- D6 s10 unfold t definition -> a292
   |     |     |     |     `- D5 s11 nonzero half-cosine -> a301
   |     |     |     `- D4 s12 sine-square denominator form -> a324
   |     |     |- D3 s13 tangent double-angle identity -> a327
   |     |     `- D3 s14 combine reciprocal-cosine and tangent -> a339
   |     `- D2 s21 solve algebraic equation for t -> a344
   |- D1 s22 rational value of m
   |  `- a389 skeleton
   |     `- D2 s23 transform second hypothesis
   |        `- a423 skeleton
   |           |- D3 s27 cotangent-to-ratio identity -> a424
   |           `- D3 s28 half-angle reciprocal identity
   |              `- a476 skeleton
   |                 |- D4 s29 numerator half-angle identity -> a477
   |                 |- D4 s30 sine double-angle identity -> a485
   |                 `- D4 s31 reciprocal tangent identity -> a487
   |- D1 s33 numerator of m -> a495
   `- D1 s34 denominator of m -> a499
```

.1.2 Search Efficiency

The search expanded 36 candidate actions at the root state (`state_0`) and at the half-angle state (`state_1`) before finding the committed skeleton decomposition. This cost reflects two sources of exploration.

- **Syntactic and strategic variation:** the model tried alternative proof paths, including direct trigonometric identities, algebraic factorizations, and half-angle sine parameterizations.
- **Lean elaboration constraints:** many candidate skeletons failed quickly during verification because of type-checking errors, invalid bindings, or missing side conditions.

The priority scoring heuristic bounded this exploration: the run reached depth 7 while keeping the active search focused on states that continued to expose useful scaffold structure.

.1.3 Local Tactic Coordination and Non-Degeneracy Proofs

While skeleton actions establish the global logical hierarchy, local tactic actions prove the leaf subgoals:

- **Algebraic Solvers:** State `state_21` was closed by `action_344` using a combination of `field_simp` and `linarith` in 6 lines of code. `state_14` was proved by `action_339` using `by_cases h : 1 + t = 0` to handle potential division-by-zero singularities.
- **Non-Degeneracy Proofs by Contradiction:** The critical division-by-zero prevention subgoals (`state_8` and `state_11`) were solved by `action_265` and `action_301` using contradiction. For instance, `action_301` introduced the hypothesis $h_{\text{cos\_half}} : \cos(x/2) = 0$, derived $\cos x = -1$ and $\tan x = 0$ via double-angle identities, and showed that this contradicts the main hypothesis $1/\cos x + \tan x = 22/7$ (since $-1 + 0 = -1 \neq 22/7$).

.1.4 Complete Reconstructed Lean 4 Proof

Below is the complete, 114-line formal proof of `aime_1991_p9` reconstructed and verified by the Lean 4 compiler. Note that since the surrounding parent theorem opens the namespaces via `open BigOperators Real Nat Rat Finset Topology`, the identifier `cos` and `Real.cos` are fully interchangeable and resolve cleanly to the same mathematical function during Lean's elaboration. Similarly, the lemma `pow_eq_zero` successfully resolves under this context to show that $a^n = 0 \implies a = 0$ for natural $n > 0$.

```
theorem aime_1991_p9
  (x : ℝ) (m : ℚ)
  (₀h : 1/Real.cos x + Real.tan x = 22/7)
  (₁h : 1/Real.sin x + Real.cot x = m) :
  ↑m.den + m.num = 44 := by
  have h_tan_half : Real.tan (x / 2) = 15 / 29 := by
    have h_trig_identity : 1 / Real.cos x + Real.tan x = (1 +
   Real.tan (x / 2)) / (1 - Real.tan (x / 2)) := by
      let t := Real.tan (x / 2)
      have h_cos : Real.cos x = (1 - t^2) / (1 + t^2) := by
        have h_cos_double : Real.cos x = (Real.cos (x / 2))^2 -
    (Real.sin (x / 2))^2 := by
          nth_rewrite 1 ←[ ₀mul_div_cancel x two_ne_zero]
          rw [two_mul, Real.cos_add]
```

```
      ring
    have h_cos_sq_val : (Real.cos (x / 2))^2 = 1 / (1 + t
^2) := by
      have h_tan_id : 1 + t^2 = 1 / (Real.cos (x / 2))^2 :=
 by
        have h_tan_sq_id : 1 + (Real.tan (x / 2))^2 = 1 / (
Real.cos (x / 2))^2 := by
          have h_sec_sq_id : (Real.cos (x / 2))^2 ≠ 0 := by
            intro h_cos_sq_zero
            have h_cos_half_zero : cos (x / 2) = 0 :=
pow_eq_zero h_cos_sq_zero
            have h_x_double : x = 2 * (x / 2) := by ring
            have h_cos_x : cos x = -1 := by
              rw [h_x_double, cos_two_mul, h_cos_half_zero]
              simp [sin_sq]
            have h_tan_x : tan x = 0 := by
              rw [tan_eq_sin_div_cos, h_x_double,
sin_two_mul, h_cos_half_zero]
              simp
            rw [h_cos_x, h_tan_x] at ₀h
            norm_num at ₀h
          have h_standard_identity : (Real.tan (x / 2))^2 +
 1 = 1 / (Real.cos (x / 2))^2 := by
            have h_cos_nz : Real.cos (x / 2) ≠ 0 := by
              intro h_zero
              apply h_sec_sq_id
              rw [h_zero, zero_pow (by norm_num)]
            rw [Real.tan_eq_sin_div_cos, div_pow]
            field_simp [h_cos_nz]
            rw [Real.sin_sq_add_cos_sq]
          rw [add_comm]
          exact h_standard_identity
        have h_t_def : t = Real.tan (x / 2) := by
          rfl
        rw [h_t_def, h_tan_sq_id]
      have h_cos_nz : Real.cos (x / 2) ≠ 0 := by
        intro h_cos_half
        have h_cos_x : Real.cos x = -1 := by
          rw [show x = 2 * (x / 2) by ring, Real.
cos_two_mul, h_cos_half]
          ring
        have h_sin_x : Real.sin x = 0 := by
          rw [show x = 2 * (x / 2) by ring, Real.
sin_two_mul, h_cos_half]
          ring
        have h_tan_x : Real.tan x = 0 := by
          rw [Real.tan_eq_sin_div_cos, h_sin_x, h_cos_x]
          exact zero_div (-1)
```

```
          rw [h_cos_x, h_tan_x] at ₀h
          norm_num at ₀h
        field_simp [h_cos_nz] at h_tan_id ⊢
        exact h_tan_id
      have h_sin_sq_val : (Real.sin (x / 2))^2 = t^2 / (1 + t
^2) := by
        have h_iden : (Real.sin (x / 2))^2 + (Real.cos (x /
2))^2 = 1 := sin_sq_add_cos_sq (x / 2)
        rw [h_cos_sq_val] at h_iden
        have h_nz : 1 + t^2 ≠ 0 := by nlinarith
        field_simp [h_nz] at h_iden ⊢
        linear_combination h_iden
      rw [h_cos_double, h_cos_sq_val, h_sin_sq_val]
      field_simp
    have h_tan : Real.tan x = (2 * t) / (1 - t^2) := by
      rw [show x = 2 * (x / 2) by ring, Real.tan_two_mul]
    have h_algebraic_step : 1 / ((1 - t^2) / (1 + t^2)) + (2
* t) / (1 - t^2) = (1 + t) / (1 - t) := by
      rw [one_div, inv_div, ← add_div]
      have h_sq : 1 + t^2 + 2 * t = (1 + t) * (1 + t) := by
ring
      have h_diff : 1 - t^2 = (1 - t) * (1 + t) := by ring
      rw [h_sq, h_diff]
      by_cases h : 1 + t = 0
      · simp [h]
      · exact mul_div_mul_right (1 + t) (1 - t) h
    rw [h_cos, h_tan]
    exact h_algebraic_step
  have h_eq : (1 + Real.tan (x / 2)) / (1 - Real.tan (x / 2))
 = 22 / 7 := by
    rw ←[ h_trig_identity]
    exact ₀h
  have h_algebra : ∀ t : ℝ, (1 + t) / (1 - t) = 22 / 7 → t =
15 / 29 := by
    intro t ht
    have h_ne : 1 - t ≠ 0 := by
      intro h
      have : 1 + t = 2 := by linarith
      rw [h] at ht
      simp [div_zero] at ht
      norm_num at ht
    field_simp [h_ne] at ht
    linarith
  apply h_algebra (Real.tan (x / 2)) h_eq
have h_m_val : m = 29 / 15 := by
  have h_trig_id : 1 / Real.sin x + Real.cot x = 1 / Real.tan
  (x / 2) := by
```

```
    have h_sum_frac : 1 / Real.sin x + Real.cot x = (1 + Real.
 cos x) / Real.sin x := by
      rw [Real.cot_eq_cos_div_sin, add_div]
    have h_half_angle : (1 + Real.cos x) / Real.sin x = 1 /
 Real.tan (x / 2) := by
      have h_num_exp : 1 + Real.cos x = 2 * Real.cos (x / 2)
 ^ 2 := by
        nth_rewrite 1 [show x = 2 * (x / 2) by ring]
        rw [Real.cos_two_mul, sq]
        ring
      have h_den_exp : Real.sin x = 2 * Real.sin (x / 2) *
 Real.cos (x / 2) := by
        rw ←[ Real.sin_two_mul, ₀mul_div_cancel x two_ne_zero
 ]
      have h_tan_inv : 1 / Real.tan (x / 2) = Real.cos (x /
 2) / Real.sin (x / 2) := by
        rw [tan_eq_sin_div_cos, one_div_div]
      rw [h_num_exp, h_den_exp, h_tan_inv]
      field_simp
    rw [h_sum_frac, h_half_angle]
  have h_m_real : (m : ℝ) = ((29 / 15 : ℚ) : ℝ) := by
    rw ←[ ₁h, h_trig_id, h_tan_half]
    norm_num
  exact Rat.cast_inj.mp h_m_real
have h_num : m.num = 29 := by
  rw [h_m_val]
  norm_num
have h_den : m.den = 15 := by
  rw [h_m_val]
  norm_num
rw [h_num, h_den]
norm_num
```

Appendix A

Running Example

Consider the following simple theorem:

```
theorem split_imp
(P Q R : Prop)
(hpq : P -> Q)
(hpr : P -> R) :
P -> Q /\ R := by
```

The initial goal is to prove $P \to Q \wedge R$. A local proof action may solve the goal directly:

```
intro hp
exact And.intro (hpq hp) (hpr hp)
```

If Lean accepts this proof body and no goal remains, the current proof state is solved. This action does not create child proof states. It is a complete local attempt for the current target hole.

A skeleton action may instead expose the two child subgoals Q and R:

```
intro hp
have hq : Q := by
  sorry
have hr : R := by
  sorry
exact And.intro hq hr
```

This skeleton does more than list two subgoals. It also gives the proof structure that will use the subgoals after they are solved. The declarations `hq` and `hr` are the child subgoals, and the final line

```
exact And.intro hq hr
```

explains how they are combined to prove the parent goal.

The two placeholders create two child subgoals. When the system focuses on the

first child, the scaffold keeps the surrounding proof structure, but only the proof of `hq` remains as the target:

```
intro hp
have hq : Q := by
  sorry
have hr : R := by
  admit
exact And.intro hq hr
```

The current node is responsible only for replacing the target `sorry`. The sibling subgoal `hr` is temporarily represented by `admit`, meaning that it is not solved by this node. In this scaffold, the proof of `hq` can be:

```
exact hpq hp
```

When the system focuses on the second child, the roles are reversed:

```
intro hp
have hq : Q := by
  admit
have hr : R := by
  sorry
exact And.intro hq hr
```

Now the target is the proof of `hr`, and the proof body can be:

```
exact hpr hp
```

The use of `admit` here is temporary. It allows the system to check one child proof inside the parent scaffold while the sibling child is still open. These checks are not final proofs of the theorem. They only verify whether a proposed replacement is valid in the proof context where it will later be used.

After both children are solved, GammaZero inserts the solved proof bodies back into the original skeleton:

```
intro hp
have hq : Q := by
  exact hpq hp
have hr : R := by
  exact hpr hp
```

```
exact And.intro hq hr
```

The completed scaffold is then checked again by Lean. This final check is essential: the parent proof is accepted only if the stitched proof contains no remaining `sorry` or `admit` and Lean verifies the whole block.

This example illustrates the boundary between the two action types. A local proof action tries to close the current target directly. A skeleton action introduces proof structure and may create child subgoals. Each child is solved in the scaffold where it was created, and the parent skeleton succeeds only when all required children are solved and the completed proof is accepted by Lean.

The example is simple, but the same pattern applies to larger proofs. A difficult theorem may require extra bounds, normalization steps, case-specific facts, or auxiliary lemmas that are not explicit in the theorem statement. A skeleton makes such hidden structure explicit. Instead of forcing the search to discover a long tactic sequence step by step, the system can first propose a proof structure, then solve the child subgoals inside that structure.

Appendix B

Sorrifier Examples and Case Studies

This subsection illustrates how the Sorrifier converts different Lean failures into placeholder-compilable scripts and how the line-based syntactic reward is computed. In all examples, blank lines and comments are excluded from the line count.

Case 1: Parser error. Consider a generated proof body containing a malformed tactic:

```
have h := 1
apply h (
```

Lean reports a parser error because the opening parenthesis is not closed. The Sorrifier localizes the failing tactic line and replaces it with `sorry`:

```
have h := 1
sorry
```

The original body contains two clean lines. One line survives unchanged, while the malformed tactic is replaced. Therefore, the syntactic reward is:

$$r_{\text{env}} = \frac{1}{2} = 0.5.$$

Case 2: Unresolved name and cascade cleanup. Consider a generated proof body that calls a nonexistent lemma:

```
apply nonexistent_lemma
rfl
```

Lean reports an unknown identifier error for `nonexistent_lemma`. After this failure, the following line is no longer useful in the recovered proof context. The Sorrifier therefore replaces the failed region with a single placeholder:

```
sorry
```

Although the original body has two clean lines, neither line survives unchanged in the patched body. Thus:

$$r_{\text{env}} = \frac{0}{2} = 0.0.$$

This example shows that recovery is not purely local at the textual level: an early failure may invalidate later commands that depend on the proof state created by the failed command.

Case 3: Skeleton partial recovery. The following skeleton introduces a valid proof step before producing an invalid tactic:

```
have h : 1 + 1 = 2 := by
  rfl
apply nonexistent_lemma
exact h
```

The first two lines form a valid local proof step. The third line fails because the lemma name cannot be resolved, and the final line becomes unusable after the failed application. The patched body is:

```
have h : 1 + 1 = 2 := by
  rfl
sorry
```

The original body has four clean lines. The first two lines survive unchanged, while the remaining two lines are replaced by a placeholder. Therefore:

$$r_{\text{env}} = \frac{2}{4} = 0.5.$$

This case illustrates why recovery is useful for skeleton actions: even when the generated proof does not fully close the goal, valid setup lines and child subgoals can still be preserved and rewarded.

Case 4: Type mismatch. A minimal type mismatch example is:

```
exact 1
```

when the expected target is a proposition. Lean rejects the term because a numeral does not inhabit the expected proposition. The recovered body is:

```
sorry
```

No original clean line survives, giving:

$$r_{\text{env}} = 0.0.$$